# Predictive Modeling with Linear Regression Project: ReCell

## Problem Statement

### Business Context

Buying and selling used phones and tablets used to be something that happened on a handful of online marketplace sites. But the used and refurbished device market has grown considerably over the past decade, and a new IDC (International Data Corporation) forecast predicts that the used phone market would be worth \$52.7bn by 2023 with a compound annual growth rate (CAGR) of 13.6% from 2018 to 2023. This growth can be attributed to an uptick in demand for used phones and tablets that offer considerable savings compared with new models.

Refurbished and used devices continue to provide cost-effective alternatives to both consumers and businesses that are looking to save money when purchasing one. There are plenty of other benefits associated with the used device market. Used and refurbished devices can be sold with warranties and can also be insured with proof of purchase. Third-party vendors/platforms, such as Verizon, Amazon, etc., provide attractive offers to customers for refurbished devices. Maximizing the longevity of devices through second-hand trade also reduces their environmental impact and helps in recycling and reducing waste. The impact of the COVID-19 outbreak may further boost this segment as consumers cut back on discretionary spending and buy phones and tablets only for immediate needs.

### Objective

The rising potential of this comparatively under-the-radar market fuels the need for an ML-based solution to develop a dynamic pricing strategy for used and refurbished devices. ReCell, a startup aiming to tap the potential in this market, has hired you as a data scientist. They want you to analyze the data provided and build a linear regression model to predict the price of a used phone/tablet and identify factors that significantly influence it.

### Data Description

The data contains the different attributes of used/refurbished phones and tablets. The data was collected in the year 2021. The detailed data dictionary is given below.

- brand_name: Name of manufacturing brand
- os: OS on which the device runs
- screen_size: Size of the screen in cm
- 4g: Whether 4G is available or not
- 5g: Whether 5G is available or not
- main_camera_mp: Resolution of the rear camera in megapixels
- selfie_camera_mp: Resolution of the front camera in megapixels
- int_memory: Amount of internal memory (ROM) in GB
- ram: Amount of RAM in GB
- battery: Energy capacity of the device battery in mAh
- weight: Weight of the device in grams
- release_year: Year when the device model was released
- days_used: Number of days the used/refurbished device has been used
- normalized_new_price: Normalized price of a new device of the same model in euros
- normalized_used_price: Normalized price of the used/refurbished device in euros

## Importing necessary libraries

In [ ]:
```
# Installing the libraries with the specified version.
# uncomment and run the following line if Google Colab is being used
# !pip install scikit-learn==1.2.2 seaborn==0.13.1 matplotlib==3.7.1
numpy==1.25.2 pandas==1.5.3 -q --user
```

In [ ]:
```
# Installing the libraries with the specified version.
# uncomment and run the following lines if Jupyter Notebook is being used
# !pip install scikit-learn==1.2.2 seaborn==0.11.1 matplotlib==3.3.4
numpy==1.24.3 pandas==1.5.2 -q --user
```

**Note**: *After running the above cell, kindly restart the notebook kernel and run all cells sequentially from the start again.*

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
import statsmodels.api as sm
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    mean_absolute_percentage_error,r2_score
)
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

## Loading the dataset

In [2]:
```
from google.colab import drive
drive.mount('/content/drive')

df = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/used_device_data.csv')

df.head()
```

```
Mounted at /content/drive
```

Out[2]:

| | brand_name | os | screen_size | 4g | 5g | main_camera_mp | selfie_cam |
|---|---|---|---|---|---|---|---|
| **0** | Honor | Android | 14.50 | yes | no | 13.0 | 5.0 |
| **1** | Honor | Android | 17.30 | yes | yes | 13.0 | 16.0 |
| **2** | Honor | Android | 16.69 | yes | yes | 13.0 | 8.0 |
| **3** | Honor | Android | 25.50 | yes | yes | 13.0 | 8.0 |
| **4** | Honor | Android | 15.32 | yes | no | 13.0 | 8.0 |

## Data Overview

- Observations
- Sanity checks

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3454 entries, 0 to 3453
Data columns (total 15 columns):
 #   Column                 Non-Null Count  Dtype  
---  ------                 --------------  -----  
 0   brand_name             3454 non-null   object 
 1   os                     3454 non-null   object 
 2   screen_size            3454 non-null   float64
 3   4g                     3454 non-null   object 
 4   5g                     3454 non-null   object 
 5   main_camera_mp         3275 non-null   float64
 6   selfie_camera_mp       3452 non-null   float64
 7   int_memory             3450 non-null   float64
 8   ram                    3450 non-null   float64
 9   battery                3448 non-null   float64
 10  weight                 3447 non-null   float64
 11  release_year           3454 non-null   int64  
 12  days_used              3454 non-null   int64  
 13  normalized_used_price  3454 non-null   float64
 14  normalized_new_price   3454 non-null   float64
dtypes: float64(9), int64(2), object(4)
memory usage: 404.9+ KB
```

In [4]:
```
df.describe()
```

Out[4]:

| | screen_size | main_camera_mp | selfie_camera_mp | int_memory | |
|---|---|---|---|---|---|
| **count** | 3454.000000 | 3275.000000 | 3452.000000 | 3450.000000 | 3450.00 |
| **mean** | 13.713115 | 9.460208 | 6.554229 | 54.573099 | 4.03612 |
| **std** | 3.805280 | 4.815461 | 6.970372 | 84.972371 | 1.36510 |
| **min** | 5.080000 | 0.080000 | 0.000000 | 0.010000 | 0.02000 |
| **25%** | 12.700000 | 5.000000 | 2.000000 | 16.000000 | 4.00000 |
| **50%** | 12.830000 | 8.000000 | 5.000000 | 32.000000 | 4.00000 |
| **75%** | 15.340000 | 13.000000 | 8.000000 | 64.000000 | 4.00000 |
| **max** | 30.710000 | 48.000000 | 32.000000 | 1024.000000 | 12.0000 |

In [5]:
```
df.shape
```

Out[5]: (3454, 15)

In [6]:
```
# have some missing values especially main_camera_mp
df.isnull().sum()
```

Out[6]:

| | 0 |
|---|---|
| **brand_name** | 0 |
| **os** | 0 |
| **screen_size** | 0 |
| **4g** | 0 |
| **5g** | 0 |
| **main_camera_mp** | 179 |
| **selfie_camera_mp** | 2 |
| **int_memory** | 4 |
| **ram** | 4 |
| **battery** | 6 |
| **weight** | 7 |
| **release_year** | 0 |
| **days_used** | 0 |
| **normalized_used_price** | 0 |
| **normalized_new_price** | 0 |

**dtype:** int64

In [7]:
```
df.tail()
```

Out[7]:

| | brand_name | os | screen_size | 4g | 5g | main_camera_mp | selfie_c |
|---|---|---|---|---|---|---|---|
| **3449** | Asus | Android | 15.34 | yes | no | NaN | 8.0 |
| **3450** | Asus | Android | 15.24 | yes | no | 13.0 | 8.0 |
| **3451** | Alcatel | Android | 15.80 | yes | no | 13.0 | 5.0 |
| **3452** | Alcatel | Android | 15.80 | yes | no | 13.0 | 5.0 |
| **3453** | Alcatel | Android | 12.83 | yes | no | 13.0 | 5.0 |

In [8]:
```
# no dupe values
df.duplicated().sum()
```

Out[8]:
```
np.int64(0)
```

# Exploratory Data Analysis (EDA)

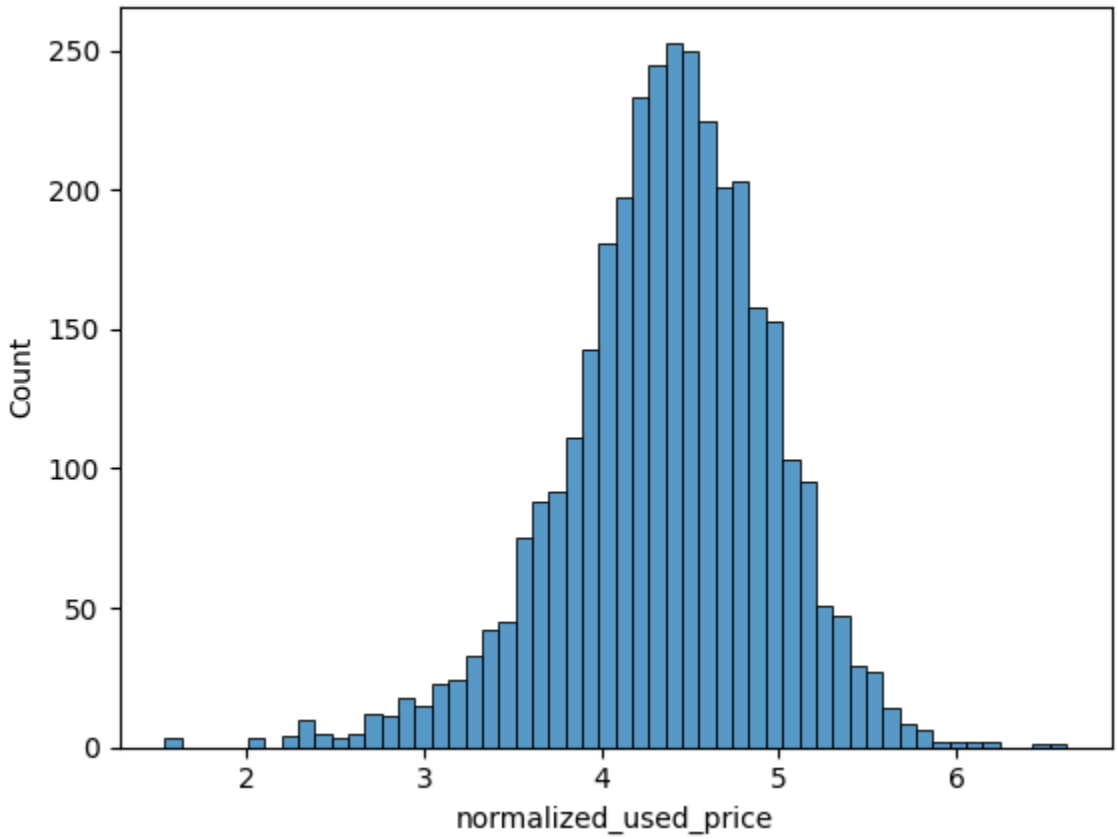
- EDA is an important part of any project involving data.
- It is important to investigate and understand the data better before building a model with it.
- A few questions have been mentioned below which will help you approach the analysis in the right manner and generate insights from the data.
- A thorough analysis of the data, in addition to the questions mentioned below, should be done.

**Questions**:

1. What does the distribution of normalized used device prices look like?
2. What percentage of the used device market is dominated by Android devices?
3. The amount of RAM is important for the smooth functioning of a device. How does the amount of RAM vary with the brand?
4. A large battery often increases a device's weight, making it feel uncomfortable in the hands. How does the weight vary for phones and tablets offering large batteries (more than 4500 mAh)?
5. Bigger screens are desirable for entertainment purposes as they offer a better viewing experience. How many phones and tablets are available across different brands with a screen size larger than 6 inches?
6. A lot of devices nowadays offer great selfie cameras, allowing us to capture our favorite moments with loved ones. What is the distribution of devices offering greater than 8MP selfie cameras across brands?
7. Which attributes are highly correlated with the normalized price of a used device?

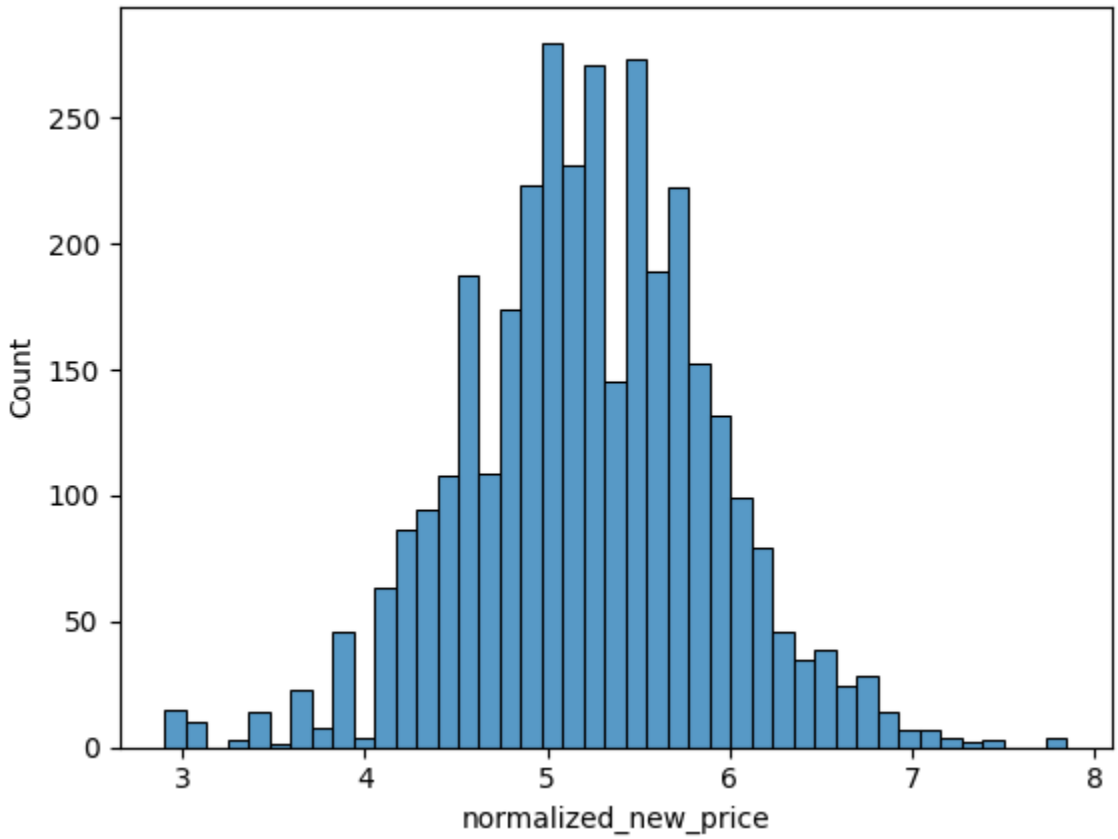
In [9]:
```
# slightly left skewed but mostly normal
sns.histplot(df['normalized_used_price'])
```

Out[9]: <Axes: xlabel='normalized_used_price', ylabel='Count'>



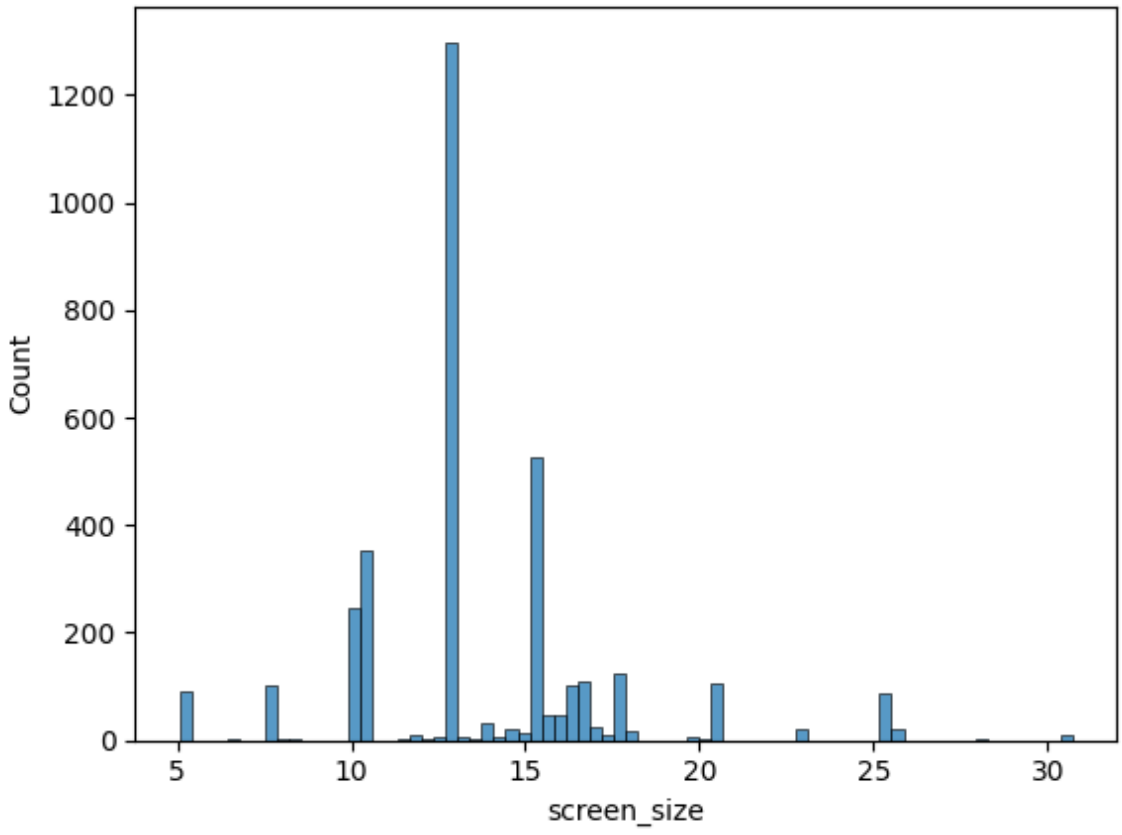

In [10]:
```
# mostly normal
sns.histplot(df['normalized_new_price'])
```

Out[10]: <Axes: xlabel='normalized_new_price', ylabel='Count'>



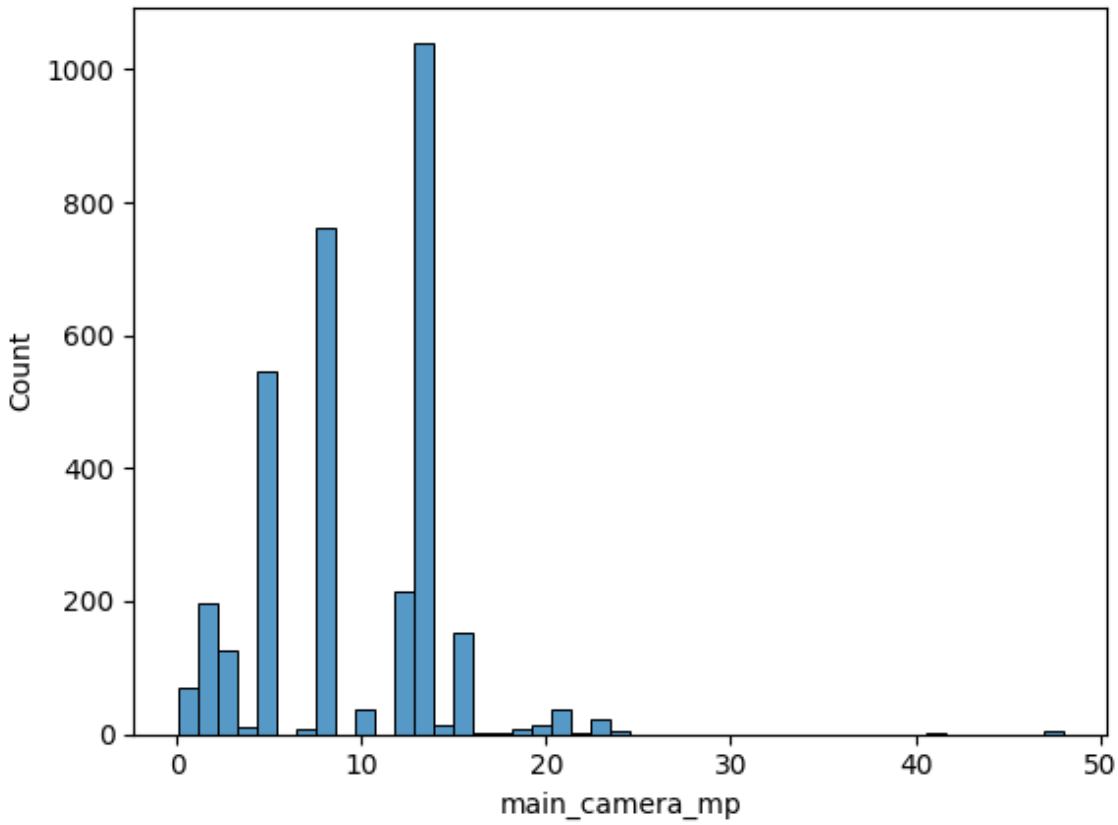

In [11]:
```
sns.histplot(df['screen_size'])
```

Out[11]: <Axes: xlabel='screen_size', ylabel='Count'>



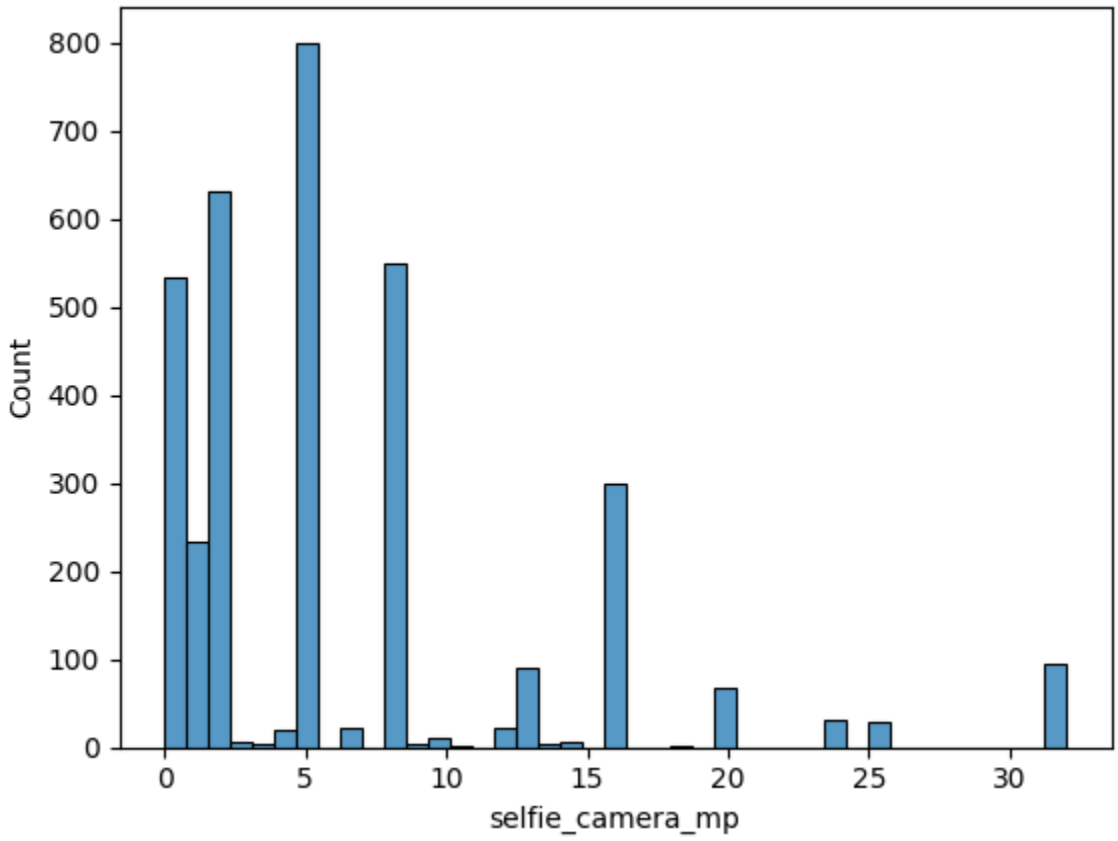
In [12]:
```
sns.histplot(df['main_camera_mp'])
```

Out[12]: <Axes: xlabel='main_camera_mp', ylabel='Count'>



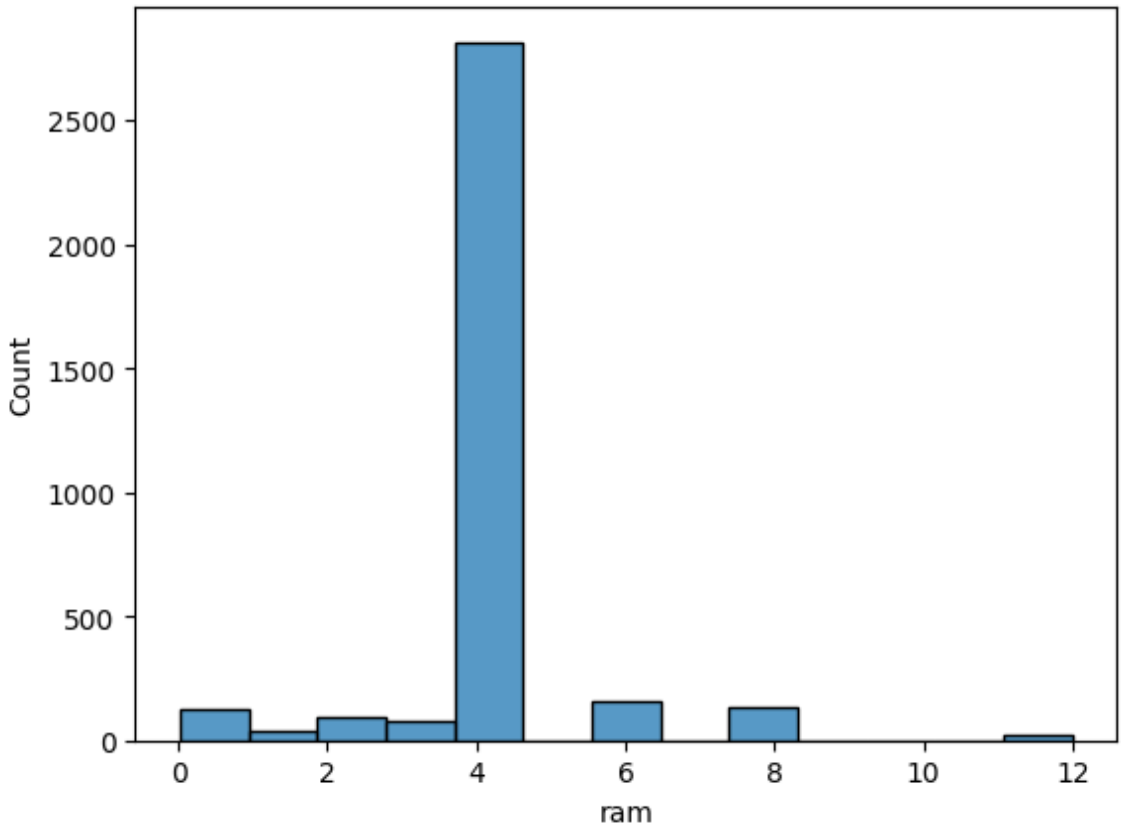

In [13]:
```
sns.histplot(df['selfie_camera_mp'])
```

Out[13]: <Axes: xlabel='selfie_camera_mp', ylabel='Count'>



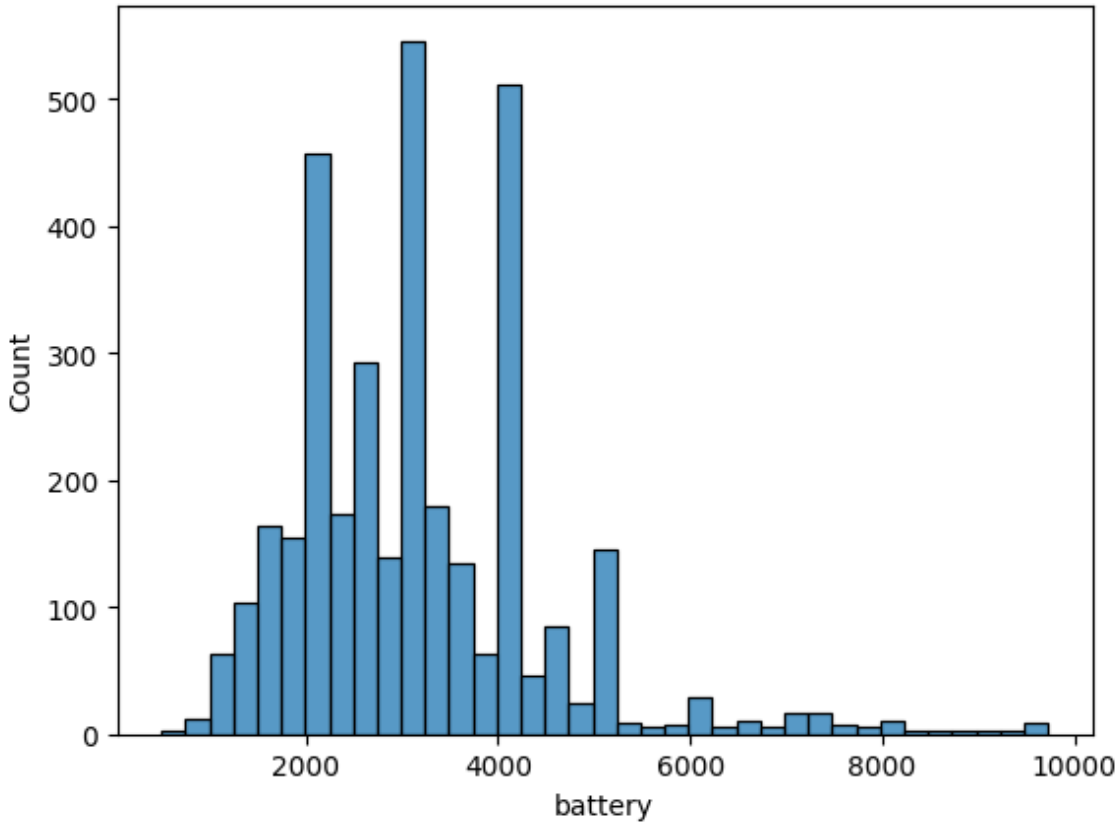

In [14]:
```
sns.histplot(df['ram'])
```

Out[14]: <Axes: xlabel='ram', ylabel='Count'>



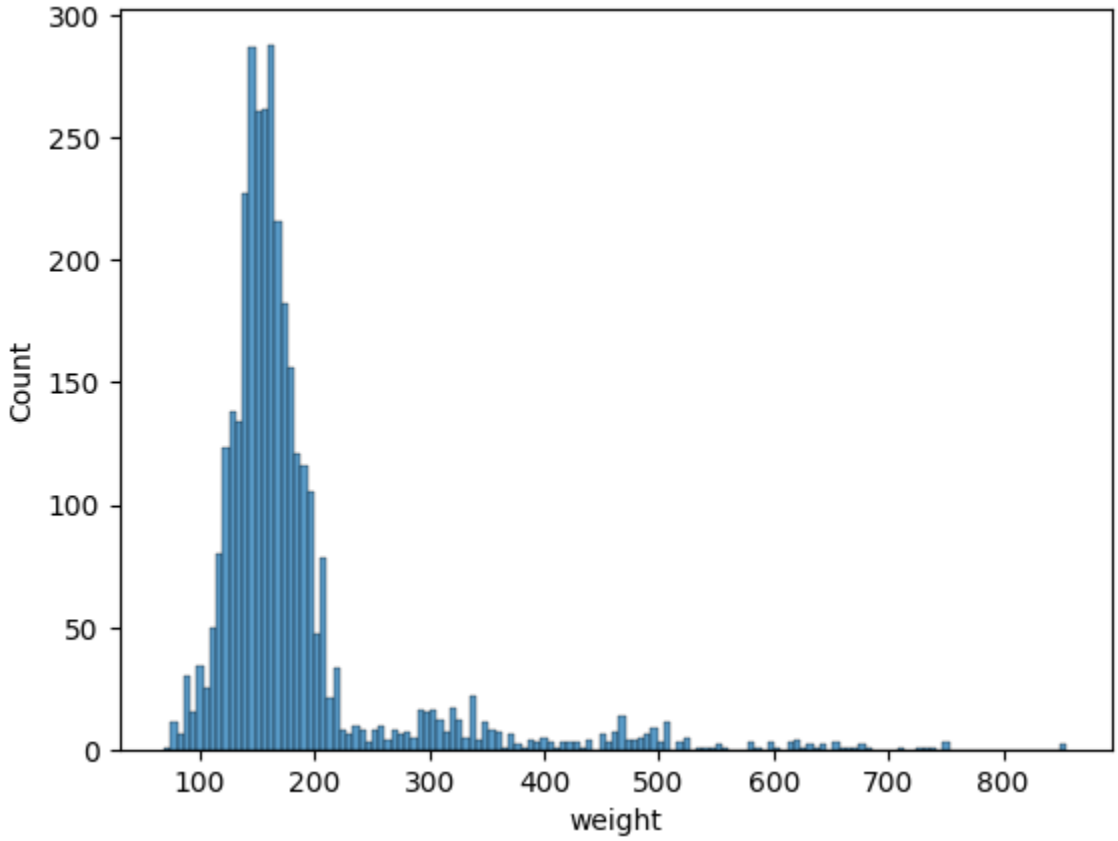
In [15]:
```
sns.histplot(df['battery'])
```

Out[15]: <Axes: xlabel='battery', ylabel='Count'>



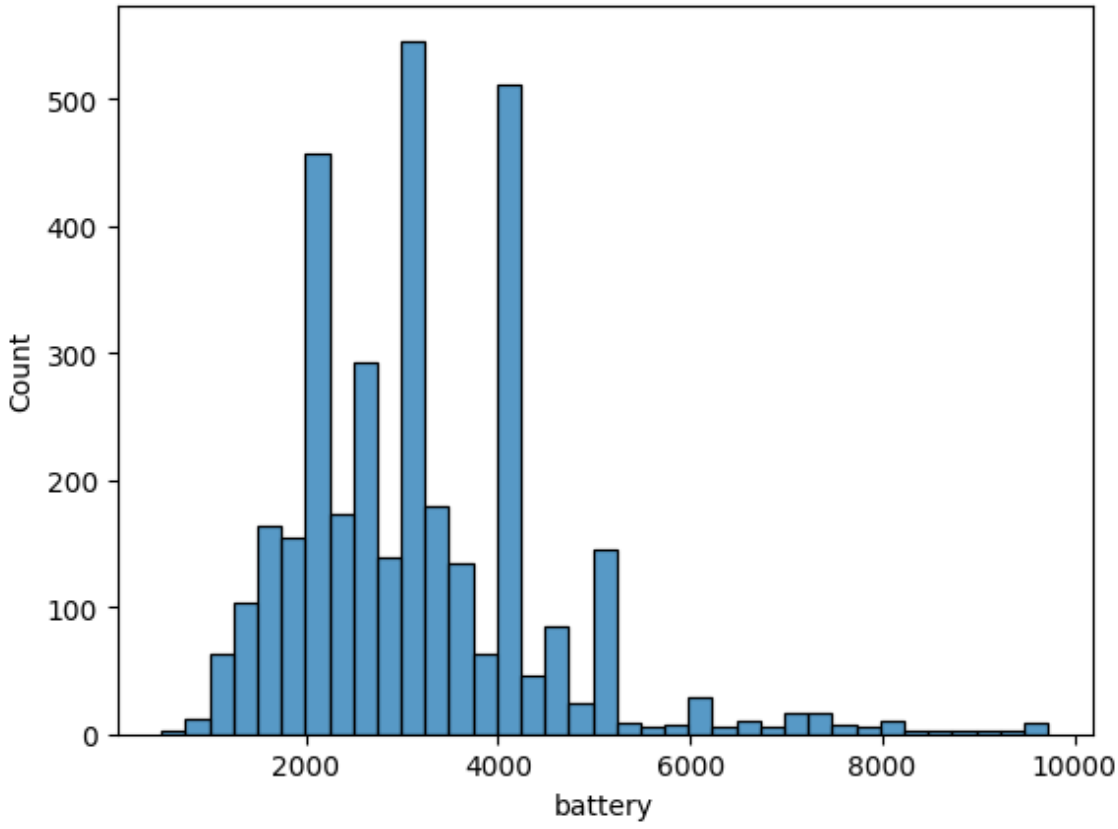

In [16]:
```
sns.histplot(df['weight'])
```

Out[16]: <Axes: xlabel='weight', ylabel='Count'>



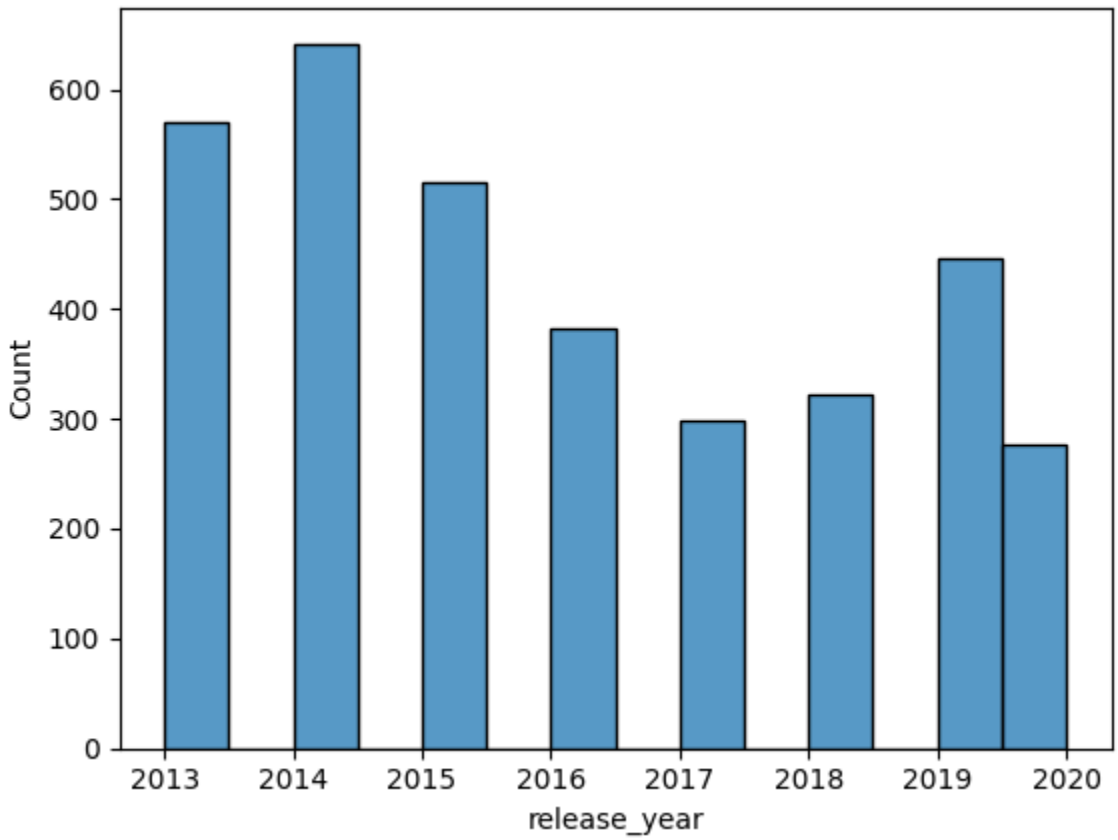

In [17]:
```
sns.histplot(df['battery'])
```

Out[17]: <Axes: xlabel='battery', ylabel='Count'>



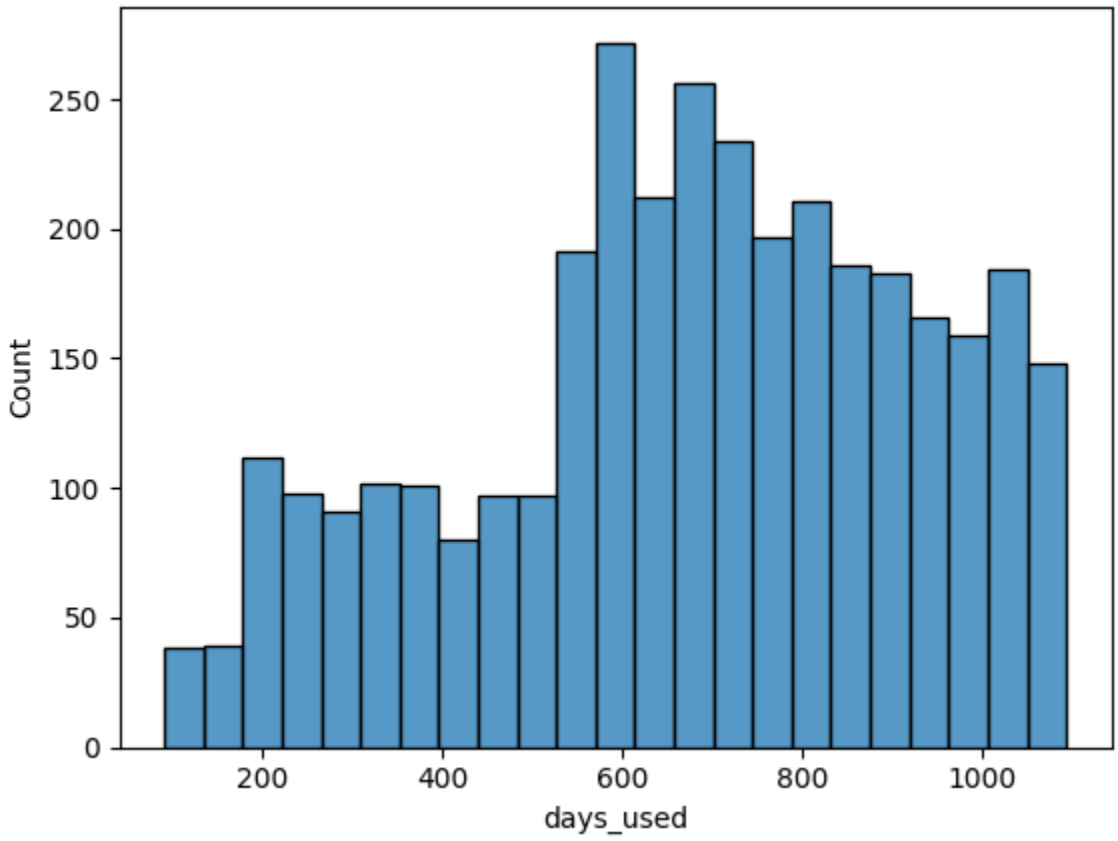
In [18]:
```
sns.histplot(df['release_year'])
```

Out[18]: <Axes: xlabel='release_year', ylabel='Count'>



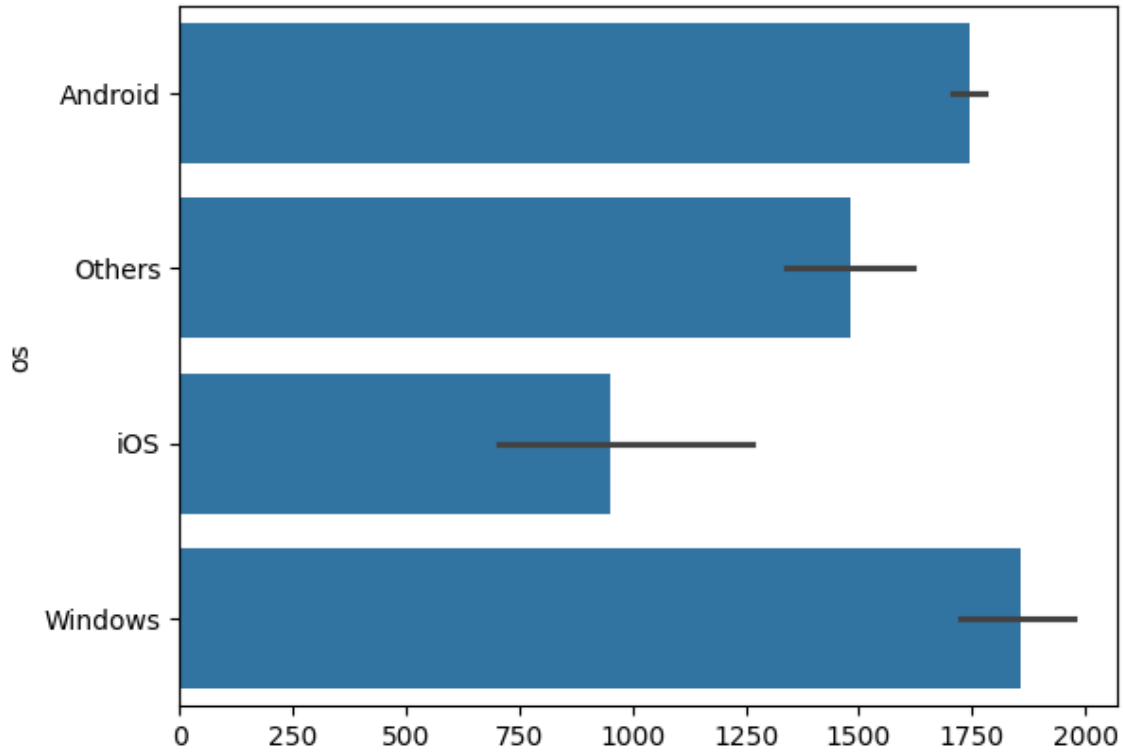

In [19]:
```
sns.histplot(df['days_used'])
```

Out[19]: <Axes: xlabel='days_used', ylabel='Count'>



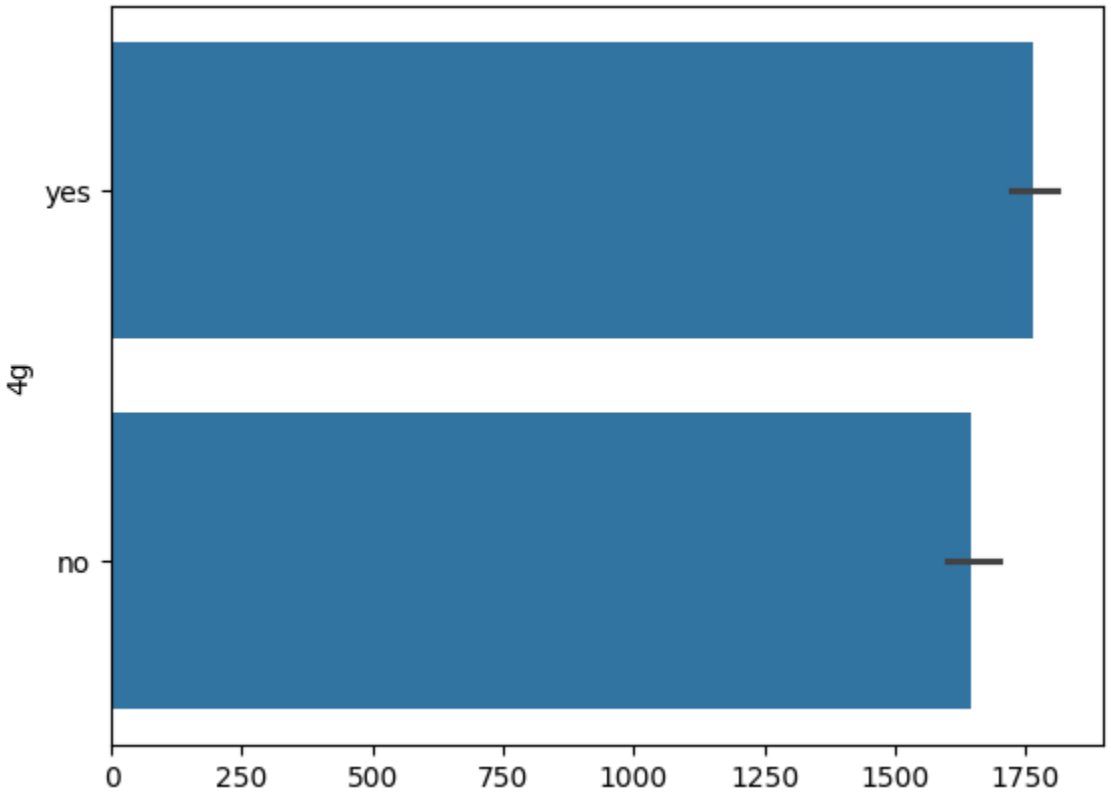

In [20]:
```
sns.barplot(df['os'])
```

Out[20]: <Axes: ylabel='os'>



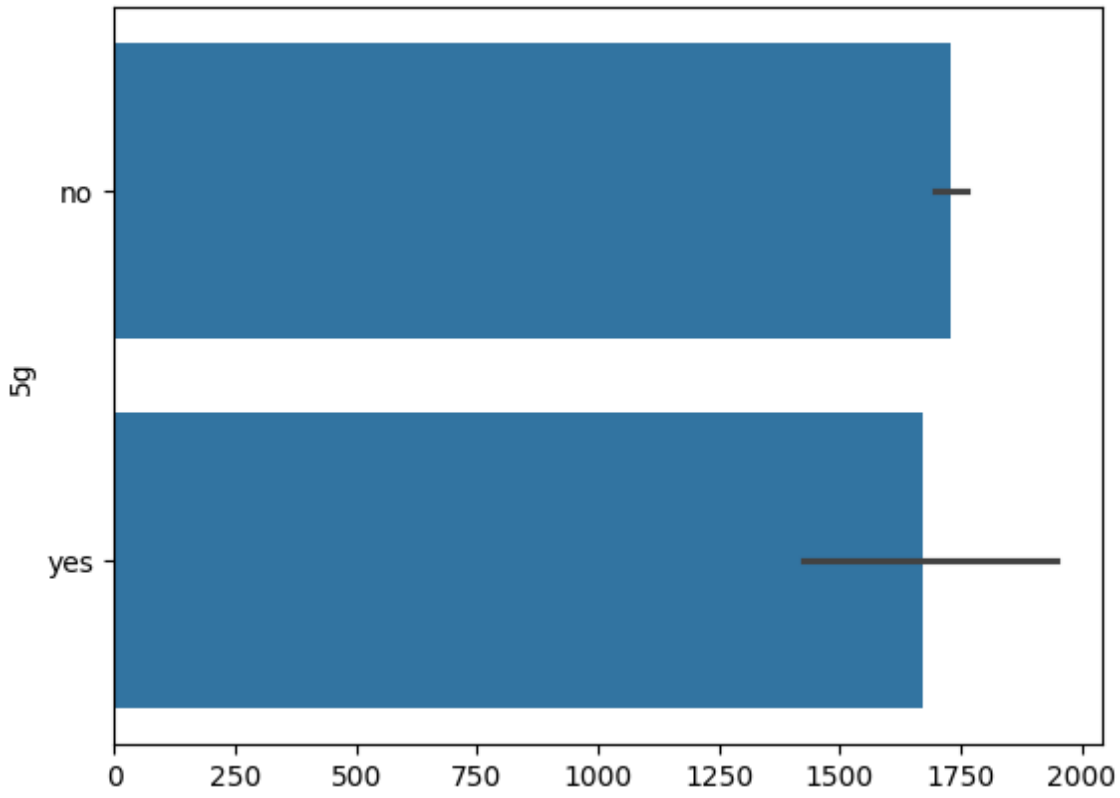
In [21]:
```
sns.barplot(df['4g'])
```

Out[21]: <Axes: ylabel='4g'>



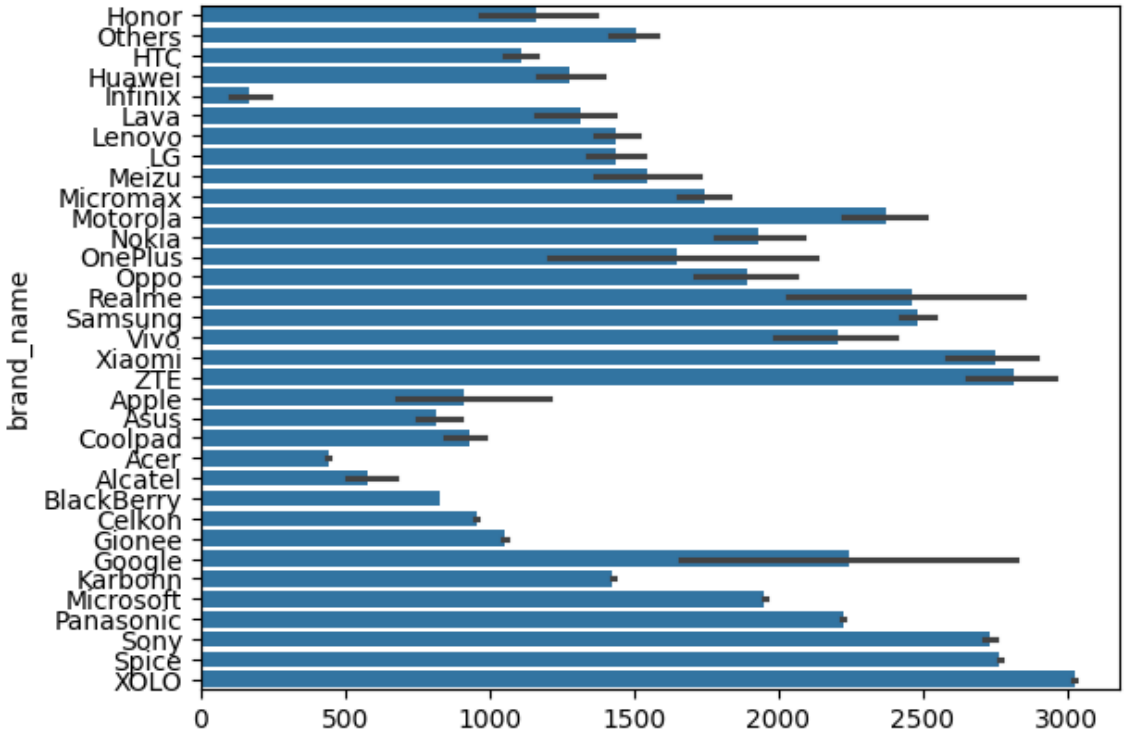

In [22]:
```
sns.barplot(df['5g'])
```

Out[22]: <Axes: ylabel='5g'>




In [23]:
```
sns.barplot(df['brand_name'])
```

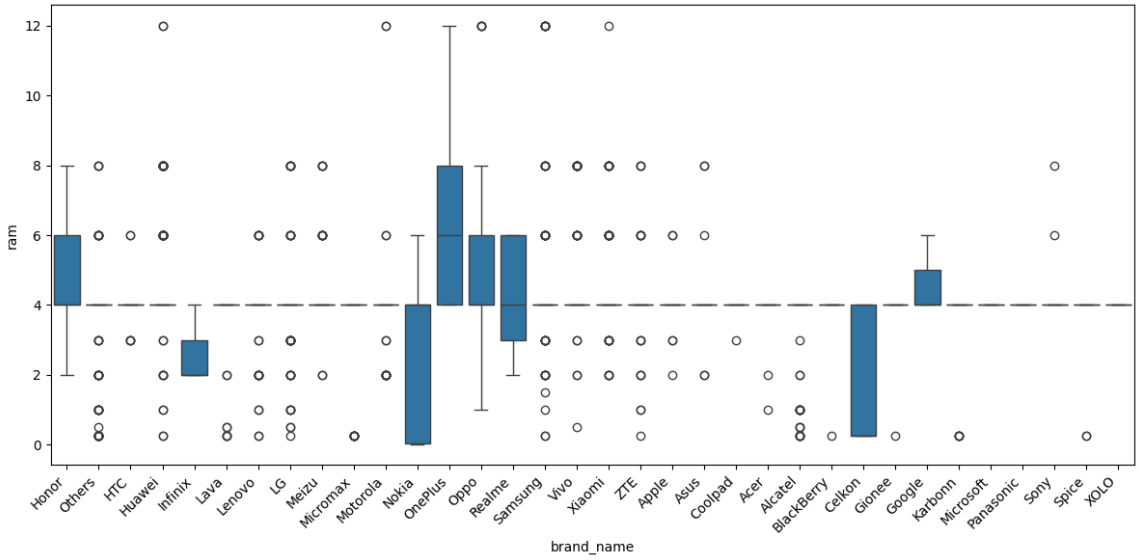
Out[23]: <Axes: ylabel='brand_name'>




In [24]:
```
# 93% of market is has Android os
df['os'].value_counts().get('Android') / df.value_counts('os').sum()
```

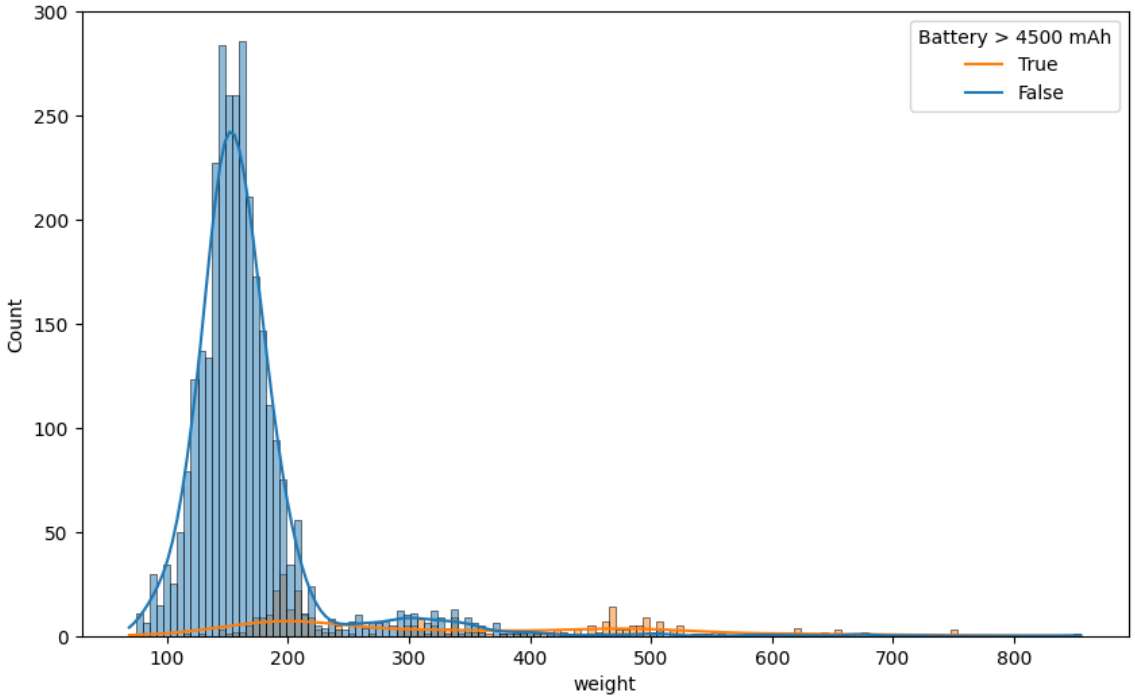
Out[24]: np.float64(0.9305153445280834)

In [25]:
```
#ram seems to vary widely amoung all brands
#Nokia and celkon have lower average ram and one plus has a higher average ram at 6 gb
plt.figure(figsize=(12, 6))
sns.boxplot(x='brand_name', y='ram', data=df)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```




In [26]:
```
#weight varies a lot with a battery greater than 4500 mAh, most fall into the 200 and 500 range while there are some large outliers

plt.figure(figsize=(10, 6))
sns.histplot(x='weight', hue=df['battery'] > 4500, data=df,kde = True)
plt.legend(title='Battery > 4500 mAh', labels=['True', 'False'])
```

Out[26]: <matplotlib.legend.Legend at 0x7b11c9549fa0>




In [27]:
```
#samsung has the most phones and tablets with a screen size large than 6 inches while infinix has the least
df[df['screen_size'] > 6].value_counts('brand_name')
```

Out[27]:

| | count |
|---|---|
| **brand_name** | |
| **Others** | 479 |
| **Samsung** | 334 |
| **Huawei** | 251 |
| **LG** | 197 |
| **Lenovo** | 171 |
| **ZTE** | 140 |
| **Xiaomi** | 132 |
| **Oppo** | 129 |
| **Asus** | 122 |
| **Vivo** | 117 |
| **Honor** | 116 |
| **Alcatel** | 115 |
| **HTC** | 110 |
| **Micromax** | 108 |

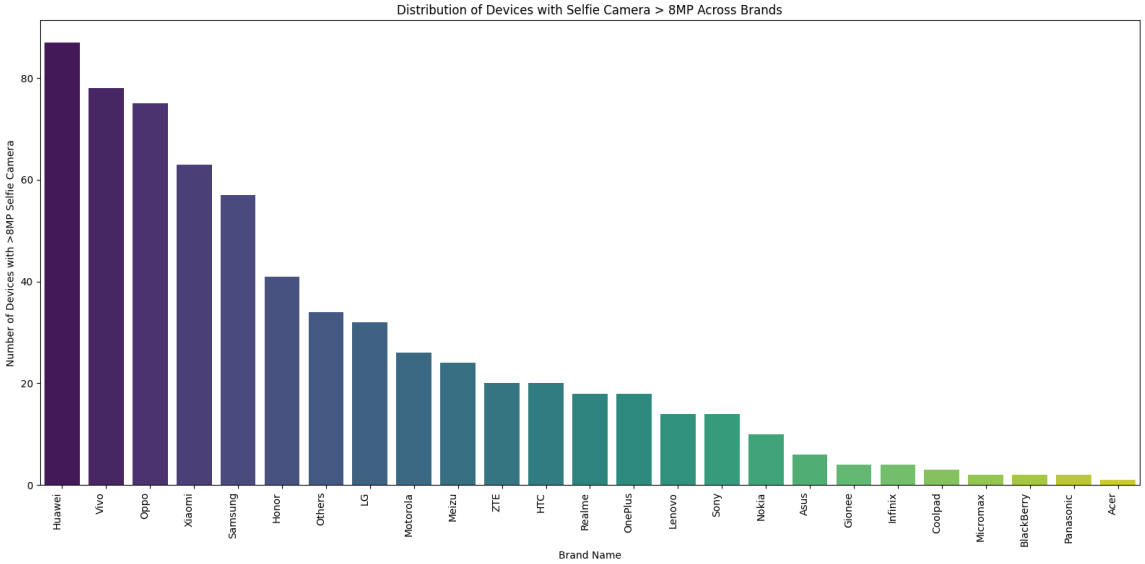
| | count |
|---|---|
| **brand_name** | |
| **Motorola** | 106 |
| **Sony** | 86 |
| **Nokia** | 72 |
| **Meizu** | 62 |
| **Gionee** | 56 |
| **Acer** | 51 |
| **XOLO** | 49 |
| **Panasonic** | 47 |
| **Realme** | 41 |
| **Apple** | 39 |
| **Lava** | 36 |
| **Spice** | 30 |
| **Karbonn** | 29 |
| **Celkon** | 25 |
| **OnePlus** | 22 |
| **Coolpad** | 22 |
| **Microsoft** | 22 |
| **BlackBerry** | 21 |
| **Google** | 15 |
| **Infinix** | 10 |

**dtype:** int64

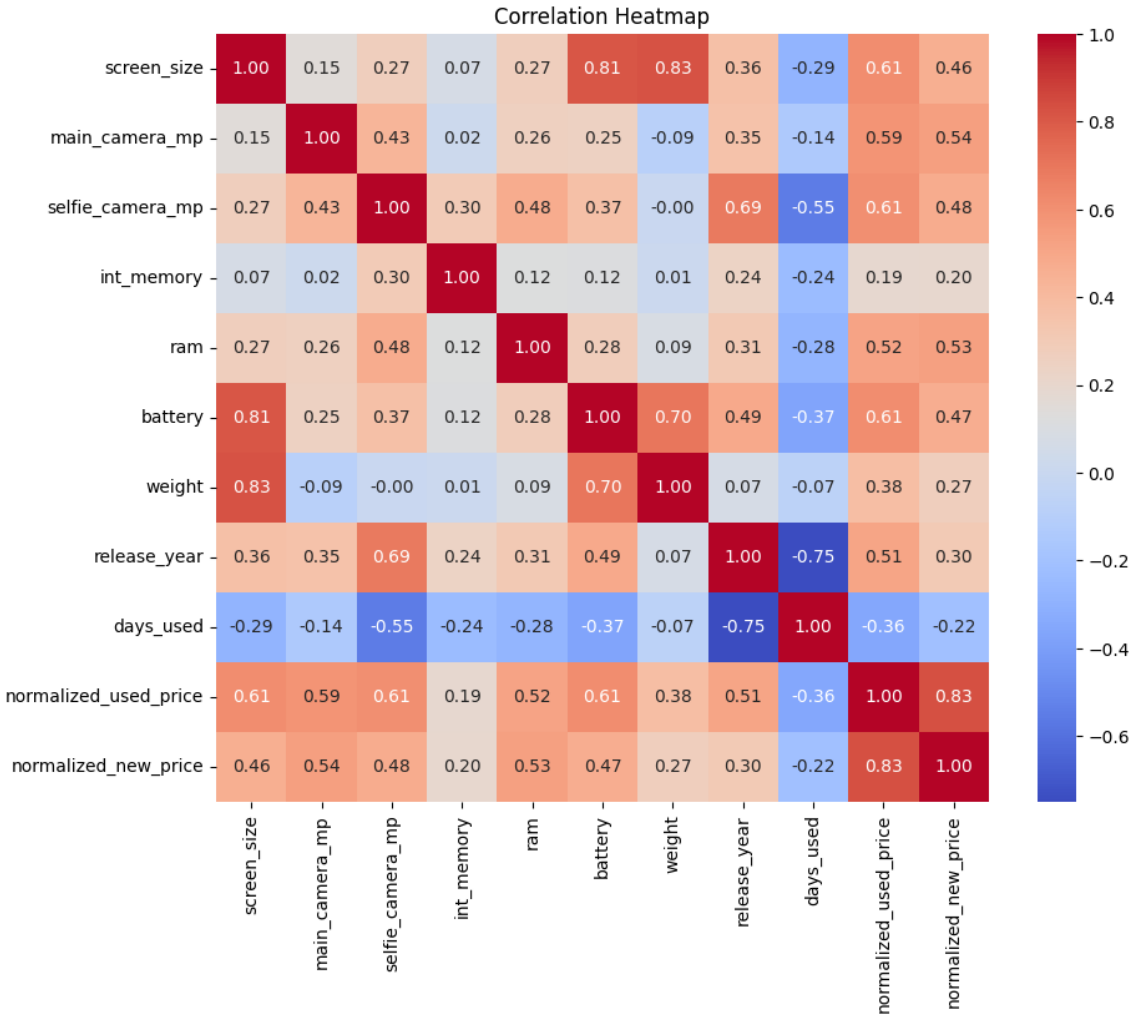
In [28]:
```
# huawei has most devices with a selfie camera mp count of greater than 8
df_filtered = df.copy()
df_filtered['selfie_camera_gt_8mp'] = df_filtered['selfie_camera_mp'] > 8

df_high_mp_selfie = df_filtered[df_filtered['selfie_camera_gt_8mp'] == True]

brand_order_by_8mp =
df_high_mp_selfie.groupby('brand_name').size().sort_values(ascending=False).in
dex

plt.figure(figsize=(16, 8))
sns.countplot(x='brand_name', data=df_high_mp_selfie,
```

```
order=brand_order_by_8mp, palette='viridis')
plt.title('Distribution of Devices with Selfie Camera > 8MP Across Brands')
plt.xlabel('Brand Name')
plt.ylabel('Number of Devices with >8MP Selfie Camera')
plt.xticks(rotation=90, ha='right')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_3970/4099259361.py:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(x='brand_name', data=df_high_mp_selfie,
order=brand_order_by_8mp, palette='viridis')
```

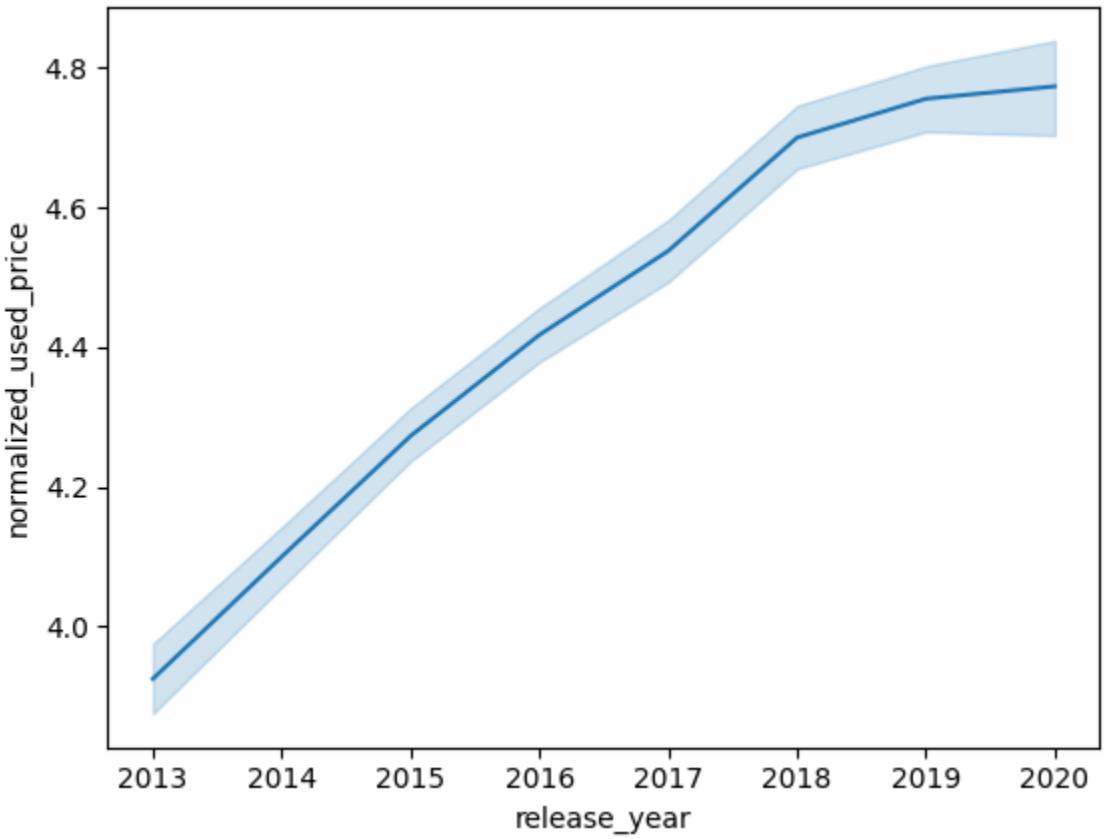



In [29]:
```
# high correlation with normalized new price, decent correlation with ram,
selfie camera, release year, and screen size
copydf = df.drop(['brand_name','os','4g','5g'],axis = 1).copy()
corr_matrix = copydf.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap')
plt.show()
```




In [30]:
```
# more recent used devices have a higher price than older phones
sns.lineplot(x='release_year', y='normalized_used_price', data=df)
```

Out[30]: <Axes: xlabel='release_year', ylabel='normalized_used_price'>




## Data Preprocessing

- Missing value treatment
- Feature engineering (if needed)
- Outlier detection and treatment (if needed)
- Preparing data for modeling
- Any other preprocessing steps (if needed)

In [31]:
```
df2 = df.copy()
```

In [32]:
```
# replacing missing values with medians of each column
for col in ['main_camera_mp', 'selfie_camera_mp', 'int_memory', 'ram',
'battery', 'weight']:
    df2[col].fillna(df2[col].median(), inplace=True)

df2.isnull().sum()
```

```
/tmp/ipykernel_3970/3123874074.py:3: FutureWarning: A value is trying
to be set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
```

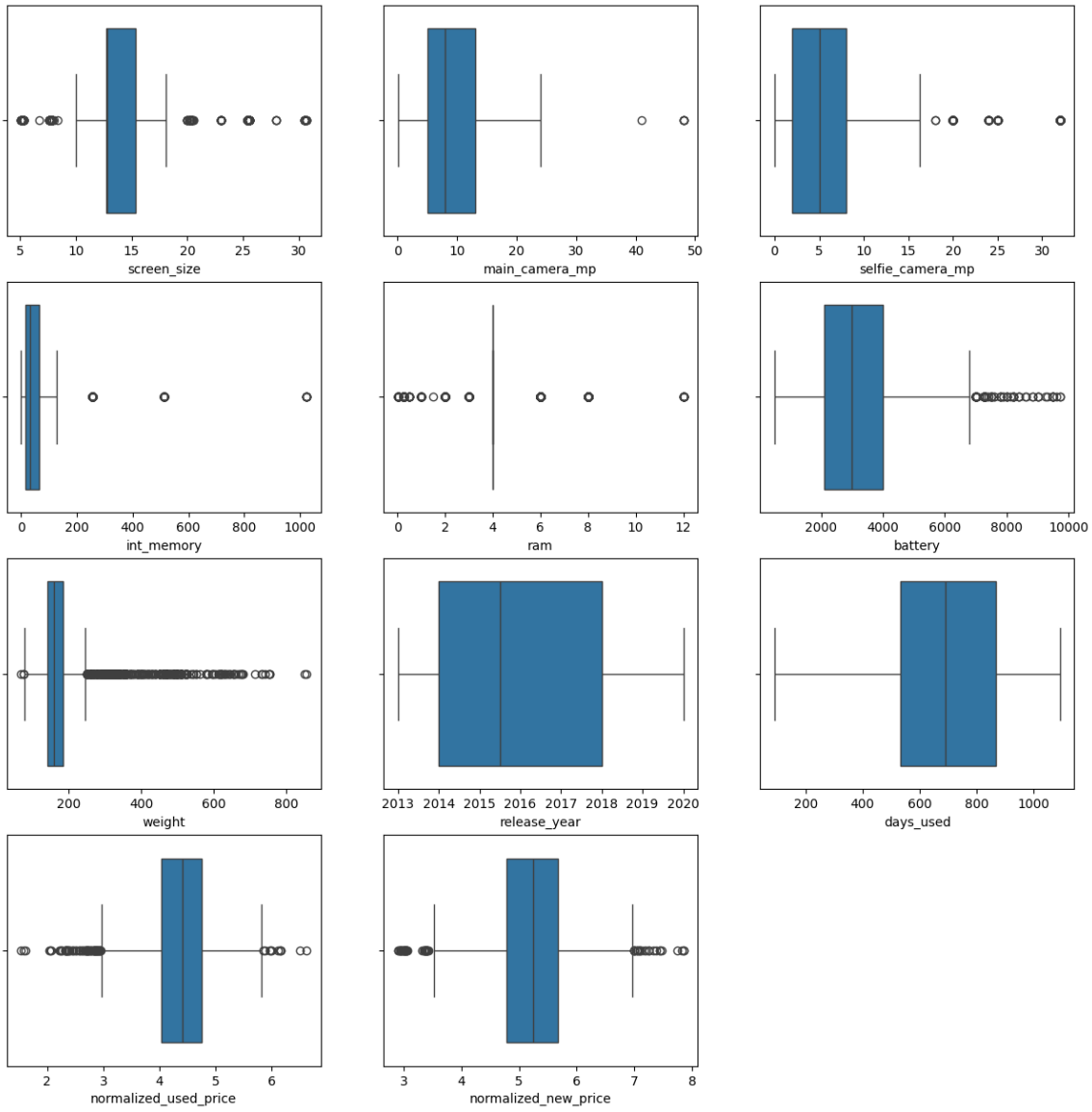
```
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df2[col].fillna(df2[col].median(), inplace=True)
```

Out[32]:

| | 0 |
|---|---|
| **brand_name** | 0 |
| **os** | 0 |
| **screen_size** | 0 |
| **4g** | 0 |
| **5g** | 0 |
| **main_camera_mp** | 0 |
| **selfie_camera_mp** | 0 |
| **int_memory** | 0 |
| **ram** | 0 |
| **battery** | 0 |
| **weight** | 0 |
| **release_year** | 0 |
| **days_used** | 0 |
| **normalized_used_price** | 0 |
| **normalized_new_price** | 0 |

**dtype:** int64

In [33]:
```
#outlier detection, outliers are normal
num_cols = df2.select_dtypes(include=np.number).columns.tolist()
plt.figure(figsize=(15, 15))
for i, variable in enumerate(num_cols):
    plt.subplot(4, 3, i + 1)
    sns.boxplot(data=df2, x=variable)
```




## Encoding Categorical Variables using One-Hot Encoding

I will now convert the nominal categorical features ( `brand_name` , `os` , `4g` , `5g` ) into numerical format using one-hot encoding. This will create new binary columns for each unique category, allowing the machine learning model to interpret these features without implying any incorrect ordinal relationships.

In [34]:
```
# one hot encoding independent columns
categorical_cols = ['brand_name', 'os', '4g', '5g']
df3 = pd.get_dummies(df2, columns=categorical_cols, drop_first=True,dtype=int)
df3.head()
```

Out[34]:

| | screen_size | main_camera_mp | selfie_camera_mp | int_memory | ram | batter |
|---|---|---|---|---|---|---|
| **0** | 14.50 | 13.0 | 5.0 | 64.0 | 3.0 | 3020.( |

| | screen_size | main_camera_mp | selfie_camera_mp | int_memory | ram | batter |
|---|---|---|---|---|---|---|
| **1** | 17.30 | 13.0 | 16.0 | 128.0 | 8.0 | 4300.0 |
| **2** | 16.69 | 13.0 | 8.0 | 128.0 | 8.0 | 4200.0 |
| **3** | 25.50 | 13.0 | 8.0 | 64.0 | 6.0 | 7250.0 |
| **4** | 15.32 | 13.0 | 8.0 | 64.0 | 3.0 | 5000.0 |

5 rows × 49 columns

In [35]:
```
# prepping model info
y = df3['normalized_used_price']
X = df3.drop(columns=['normalized_used_price', 'normalized_new_price'])
```

In [36]:
```
X = sm.add_constant(X)
```

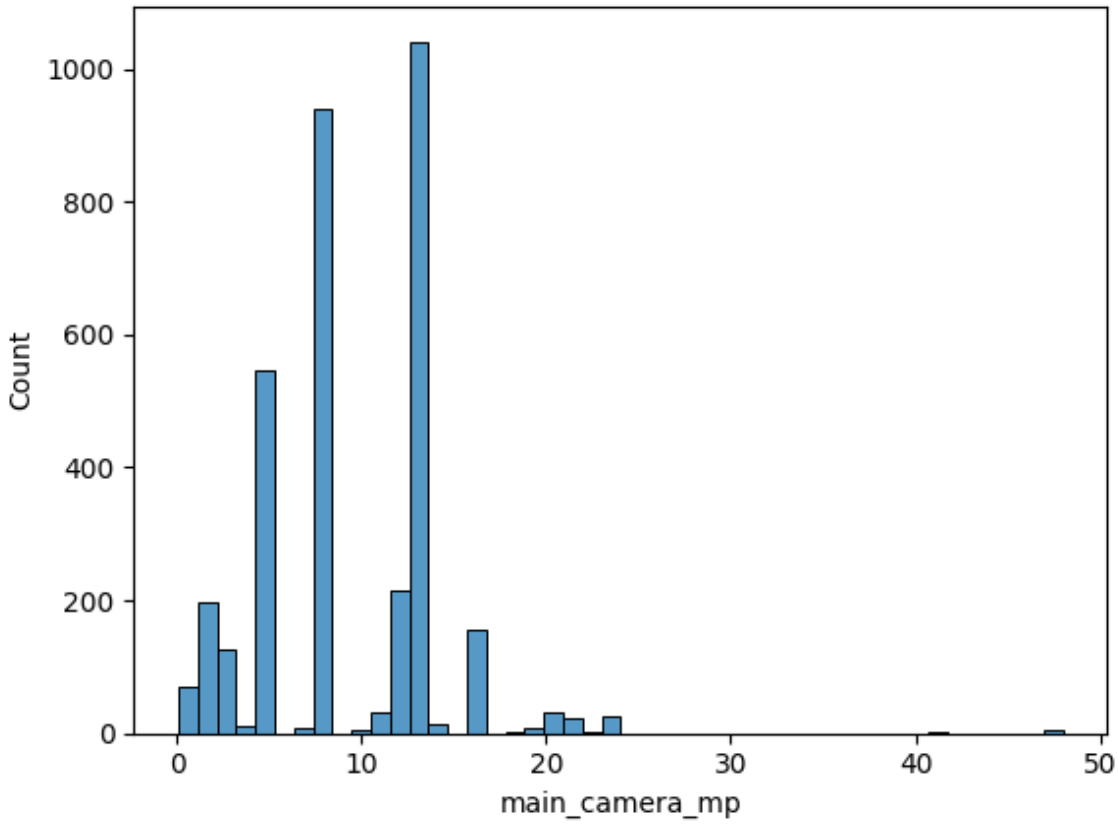
In [37]:
```
X_train,X_test,y_train,y_test = train_test_split(X,y
,test_size=0.3,random_state=0)
```

# EDA

- It is a good idea to explore the data once again after manipulating it.

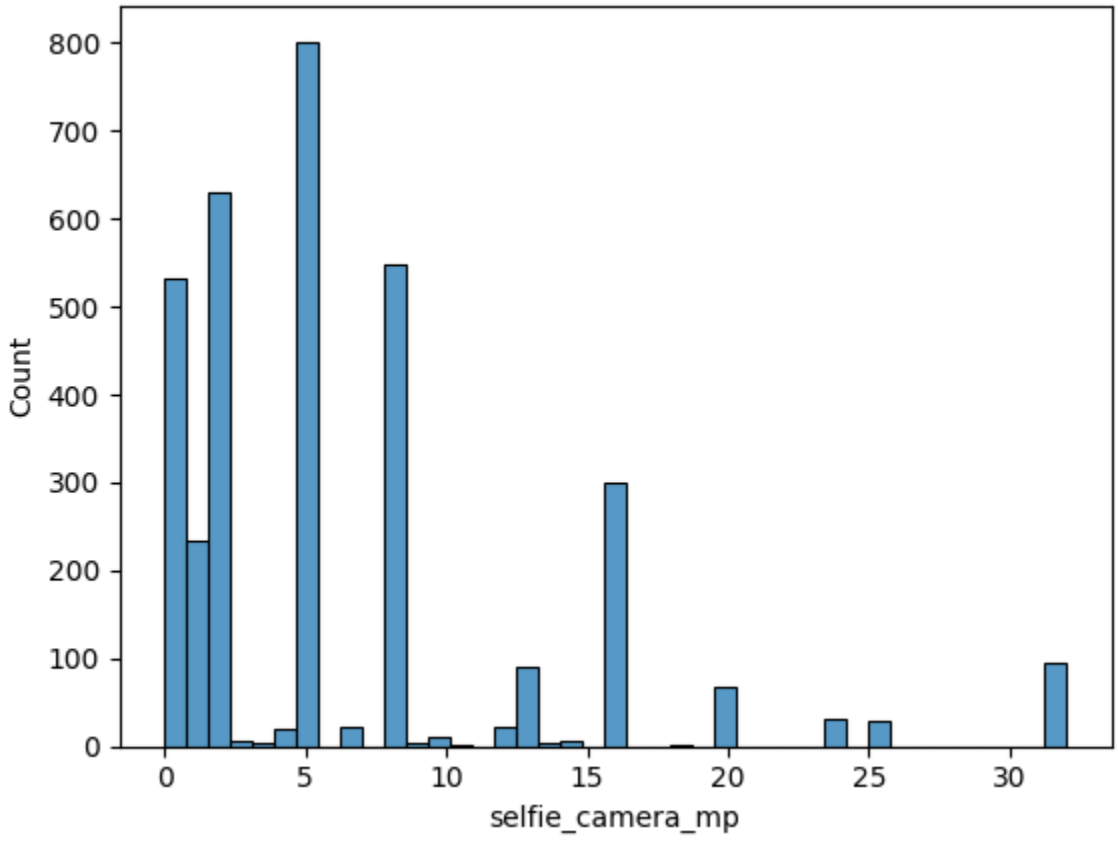
In [38]:
```
sns.histplot(df2['main_camera_mp'])
```

Out[38]: <Axes: xlabel='main_camera_mp', ylabel='Count'>



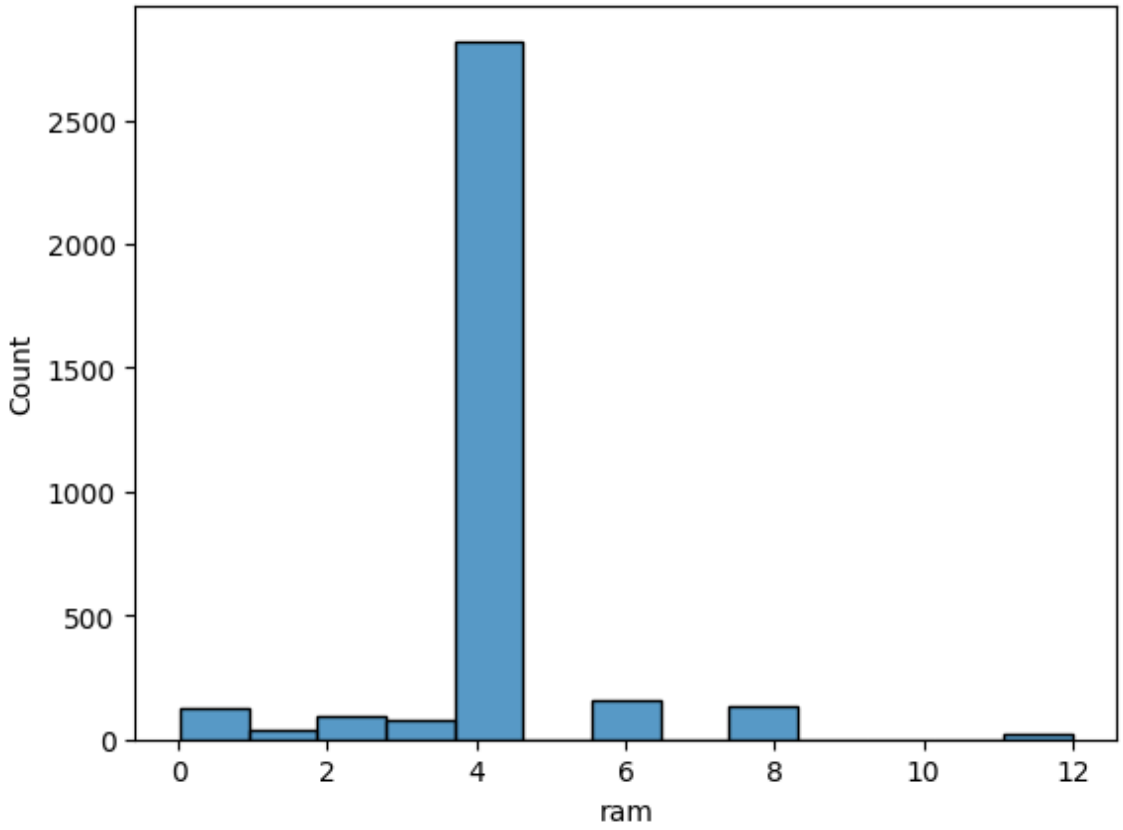

In [39]:
```
sns.histplot(df2['selfie_camera_mp'])
```

Out[39]: <Axes: xlabel='selfie_camera_mp', ylabel='Count'>



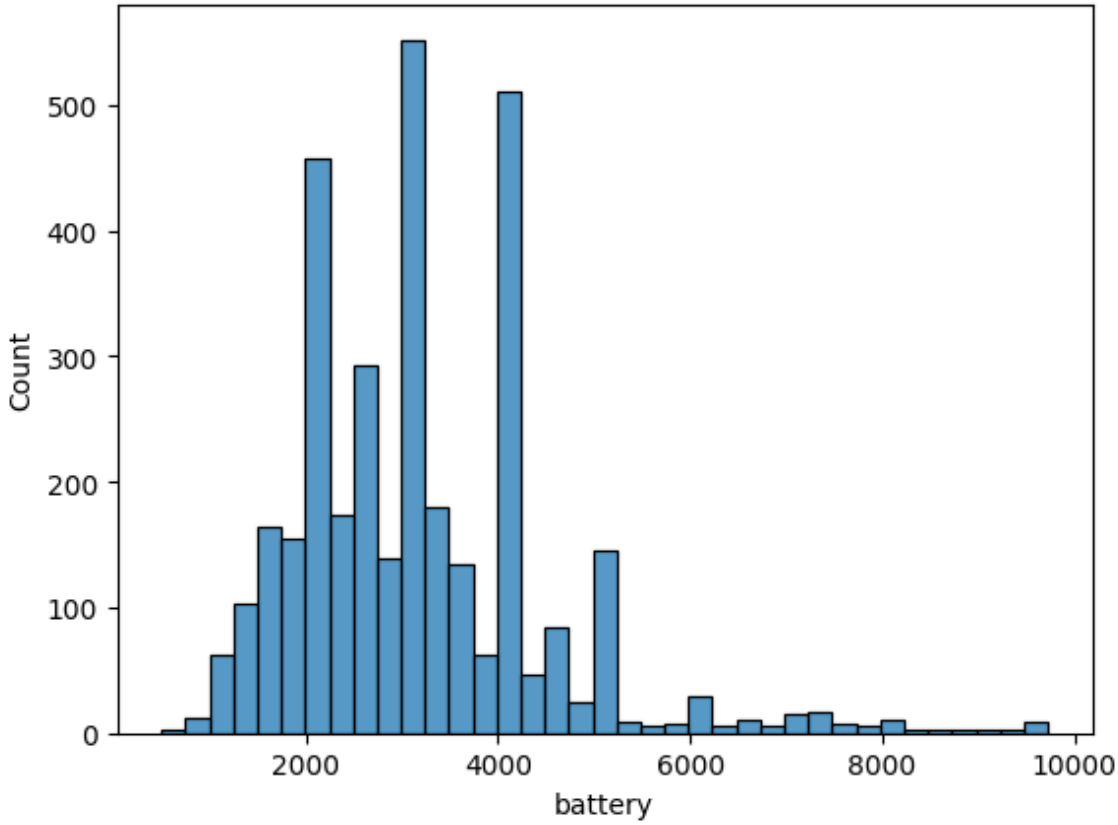
In [40]:
```
sns.histplot(df2['ram'])
```

Out[40]: <Axes: xlabel='ram', ylabel='Count'>



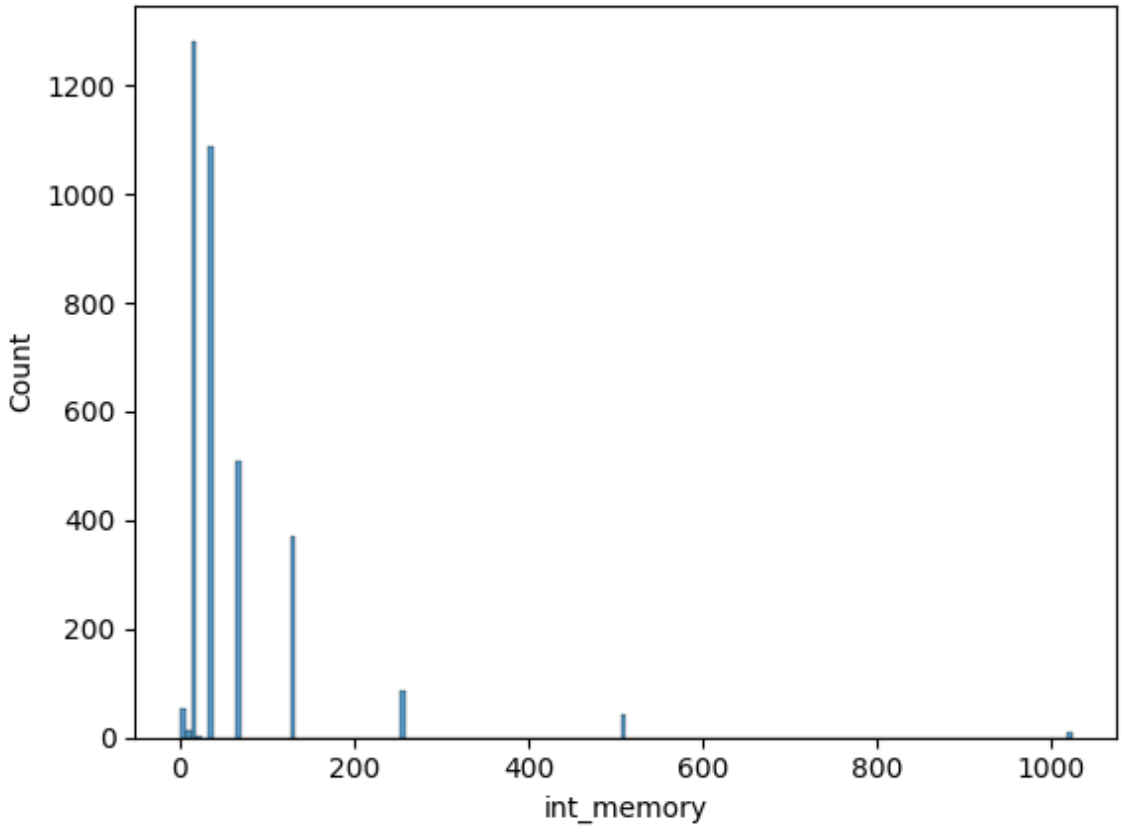

In [41]:
```
sns.histplot(df2['battery'])
```

Out[41]: <Axes: xlabel='battery', ylabel='Count'>



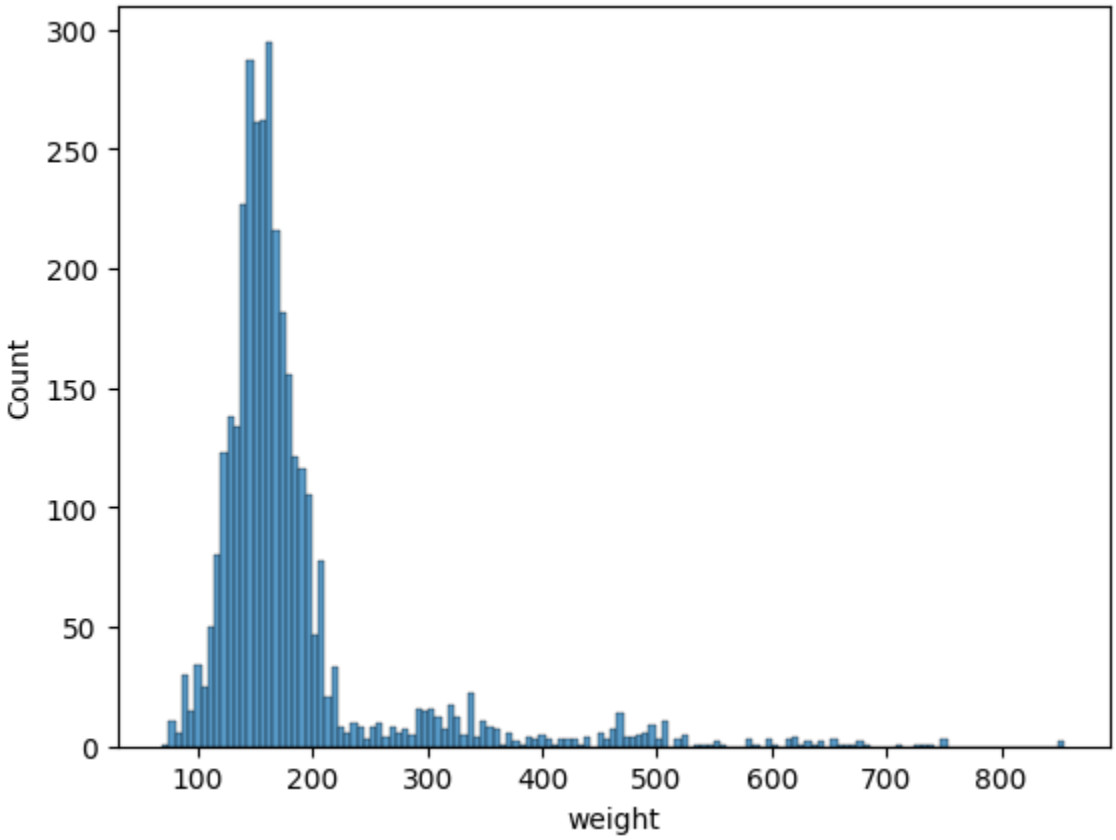

In [42]:
```
sns.histplot(df2['int_memory'])
```

Out[42]: <Axes: xlabel='int_memory', ylabel='Count'>



In [43]:
```
sns.histplot(df2['weight'])
```

Out[43]: <Axes: xlabel='weight', ylabel='Count'>




## Model Building - Linear Regression

In [44]:
```
# 76% of variance in data is explained
model = sm.OLS(y, X).fit()
model.summary()
```

Out[44]:

OLS Regression Results

| | | | |
|---|---|---|---|
| **Dep. Variable:** | normalized_used_price | **R-squared:** | 0.766 |
| **Model:** | OLS | **Adj. R-squared:** | 0.763 |
| **Method:** | Least Squares | **F-statistic:** | 237.0 |
| **Date:** | Sat, 11 Jul 2026 | **Prob (F-statistic):** | 0.00 |
| **Time:** | 18:27:51 | **Log-Likelihood:** | -564.78 |
| **No. Observations:** | 3454 | **AIC:** | 1226. |
| **Df Residuals:** | 3406 | **BIC:** | 1521. |

| | | | |
|---|---|---|---|
| **Df Model:** | 47 | | |
| **Covariance Type:** | nonrobust | | |

| | coef | std err | t | P>\|t\| | [0.025 | 0.975] |
|---|---|---|---|---|---|---|
| **const** | 34.6877 | 9.135 | 3.797 | 0.000 | 16.778 | 52.598 |
| **screen_size** | 0.0424 | 0.003 | 12.391 | 0.000 | 0.036 | 0.049 |
| **main_camera_mp** | 0.0407 | 0.001 | 28.955 | 0.000 | 0.038 | 0.043 |
| **selfie_camera_mp** | 0.0219 | 0.001 | 18.941 | 0.000 | 0.020 | 0.024 |
| **int_memory** | 0.0005 | 6.42e-05 | 7.837 | 0.000 | 0.000 | 0.001 |
| **ram** | 0.0679 | 0.005 | 13.041 | 0.000 | 0.058 | 0.078 |
| **battery** | 3.503e-06 | 7.67e-06 | 0.457 | 0.648 | -1.15e-05 | 1.85e-05 |
| **weight** | 0.0009 | 0.000 | 7.017 | 0.000 | 0.001 | 0.001 |
| **release_year** | -0.0159 | 0.005 | -3.504 | 0.000 | -0.025 | -0.007 |
| **days_used** | 3.744e-06 | 3.16e-05 | 0.119 | 0.906 | -5.82e-05 | 6.56e-05 |
| **brand_name_Alcatel** | -0.0497 | 0.048 | -1.031 | 0.302 | -0.144 | 0.045 |
| **brand_name_Apple** | 0.3654 | 0.176 | 2.071 | 0.038 | 0.019 | 0.711 |
| **brand_name_Asus** | 0.0669 | 0.048 | 1.388 | 0.165 | -0.028 | 0.161 |
| **brand_name_BlackBerry** | 0.1722 | 0.076 | 2.278 | 0.023 | 0.024 | 0.320 |
| **brand_name_Celkon** | -0.3383 | 0.067 | -5.079 | 0.000 | -0.469 | -0.208 |
| **brand_name_Coolpad** | -0.0167 | 0.074 | -0.226 | 0.821 | -0.162 | 0.128 |
| **brand_name_Gionee** | 0.0603 | 0.056 | 1.075 | 0.282 | -0.050 | 0.170 |
| **brand_name_Google** | 0.3125 | 0.085 | 3.672 | 0.000 | 0.146 | 0.479 |
| **brand_name_HTC** | 0.0734 | 0.049 | 1.491 | 0.136 | -0.023 | 0.170 |
| **brand_name_Honor** | -0.0811 | 0.050 | -1.637 | 0.102 | -0.178 | 0.016 |
| **brand_name_Huawei** | -0.0181 | 0.045 | -0.404 | 0.686 | -0.106 | 0.070 |
| **brand_name_Infinix** | 0.0008 | 0.101 | 0.007 | 0.994 | -0.197 | 0.199 |
| **brand_name_Karbonn** | -0.1416 | 0.067 | -2.105 | 0.035 | -0.274 | -0.010 |
| **brand_name_LG** | 0.0369 | 0.046 | 0.810 | 0.418 | -0.052 | 0.126 |
| **brand_name_Lava** | -0.1228 | 0.063 | -1.948 | 0.052 | -0.246 | 0.001 |
| **brand_name_Lenovo** | -0.0349 | 0.046 | -0.758 | 0.449 | -0.125 | 0.055 |

| | | | | | | |
|---|---|---|---|---|---|---|
| **brand_name_Meizu** | -0.0019 | 0.055 | -0.035 | 0.972 | -0.110 | 0.106 |
| **brand_name_Micromax** | -0.1628 | 0.048 | -3.357 | 0.001 | -0.258 | -0.068 |
| **brand_name_Microsoft** | 0.0209 | 0.086 | 0.244 | 0.807 | -0.147 | 0.189 |
| **brand_name_Motorola** | -0.0787 | 0.050 | -1.572 | 0.116 | -0.177 | 0.019 |
| **brand_name_Nokia** | 0.1345 | 0.052 | 2.600 | 0.009 | 0.033 | 0.236 |
| **brand_name_OnePlus** | 0.2697 | 0.075 | 3.612 | 0.000 | 0.123 | 0.416 |
| **brand_name_Oppo** | 0.0527 | 0.049 | 1.078 | 0.281 | -0.043 | 0.149 |
| **brand_name_Others** | -0.0084 | 0.042 | -0.199 | 0.842 | -0.092 | 0.075 |
| **brand_name_Panasonic** | -0.0577 | 0.059 | -0.985 | 0.325 | -0.172 | 0.057 |
| **brand_name_Realme** | 0.0254 | 0.062 | 0.409 | 0.682 | -0.096 | 0.147 |
| **brand_name_Samsung** | 0.0582 | 0.044 | 1.337 | 0.181 | -0.027 | 0.144 |
| **brand_name_Sony** | -0.0308 | 0.052 | -0.597 | 0.551 | -0.132 | 0.070 |
| **brand_name_Spice** | -0.1881 | 0.066 | -2.830 | 0.005 | -0.318 | -0.058 |
| **brand_name_Vivo** | -0.0214 | 0.050 | -0.431 | 0.667 | -0.119 | 0.076 |
| **brand_name_XOLO** | -0.0750 | 0.058 | -1.298 | 0.194 | -0.188 | 0.038 |
| **brand_name_Xiaomi** | -0.0006 | 0.049 | -0.013 | 0.989 | -0.096 | 0.095 |
| **brand_name_ZTE** | -0.0491 | 0.048 | -1.030 | 0.303 | -0.143 | 0.044 |
| **os_Others** | -0.1948 | 0.033 | -5.918 | 0.000 | -0.259 | -0.130 |
| **os_Windows** | -0.0288 | 0.046 | -0.623 | 0.533 | -0.119 | 0.062 |
| **os_iOS** | -0.0873 | 0.178 | -0.492 | 0.623 | -0.435 | 0.261 |
| **4g_yes** | 0.1435 | 0.016 | 8.840 | 0.000 | 0.112 | 0.175 |
| **5g_yes** | 0.1341 | 0.032 | 4.196 | 0.000 | 0.071 | 0.197 |

| | | | |
|---|---|---|---|
| **Omnibus:** | 239.042 | **Durbin-Watson:** | 1.709 |
| **Prob(Omnibus):** | 0.000 | **Jarque-Bera (JB):** | 1183.373 |
| **Skew:** | -0.051 | **Prob(JB):** | 1.08e-257 |
| **Kurtosis:** | 5.866 | **Cond. No.** | 7.38e+06 |

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 7.38e+06. This might indicate that there are strong multicollinearity or other numerical problems.

## Model Performance Check

In [45]:
```
# function to print out 5 metrics of any model
def model_performance_regression(model, predictors, target):

    # predictions
    pred = model.predict(predictors)

    # R^2
    r2 = r2_score(target, pred)

    # Adjusted R^2
    n = predictors.shape[0]   # number of observations
    k = predictors.shape[1]   # number of predictors
    adjr2 = 1 - ((1 - r2) * (n - 1) / (n - k - 1))

    # Other metrics
    rmse = np.sqrt(mean_squared_error(target, pred))
    mae = mean_absolute_error(target, pred)

    # MAPE
    mape = mean_absolute_percentage_error(target, pred) * 100

    # Create dataframe
    df_perf = pd.DataFrame(
        {
            "RMSE": [rmse],
            "MAE": [mae],
            "R-squared": [r2],
            "Adj. R-squared": [adjr2],
            "MAPE": [mape],
        }
    )

    return df_perf
```

In [46]:
```
print("Training Performance\n")
trainperf = model_performance_regression(model, X_train, y_train)
trainperf
```

```
Training Performance
```

Out[46]:

| | RMSE | MAE | R-squared | Adj. R-squared | MAPE |
|---|---|---|---|---|---|
| **0** | 0.281863 | 0.209386 | 0.75975 | 0.75488 | 5.000473 |

In [47]:
```
print("Test Performance\n")
testperf = model_performance_regression(model,X_test,y_test)
```

```
testperf
```

```
Test Performance
```

Out[47]:

| | RMSE | MAE | R-squared | Adj. R-squared | MAPE |
|---|---|---|---|---|---|
| **0** | 0.292033 | 0.218918 | 0.777644 | 0.766841 | 5.353045 |

## Checking Linear Regression Assumptions

- In order to make statistical inferences from a linear regression model, it is important to ensure that the assumptions of linear regression are satisfied.

In [48]:

```
# Calculate VIF for each independent variable

features_for_vif = X_train.drop(columns=['const']) if 'const' in
X_train.columns else X_train

vif_data = pd.DataFrame()
vif_data['feature'] = features_for_vif.columns
vif_data['VIF'] = [variance_inflation_factor(features_for_vif.values, i) for i
in range(features_for_vif.shape[1])]

display(vif_data.sort_values(by='VIF', ascending=False))
```

| | feature | VIF |
|---|---|---|
| **7** | release_year | 143.663056 |
| **0** | screen_size | 94.582510 |
| **6** | weight | 28.745211 |
| **5** | battery | 26.886630 |
| **4** | ram | 21.132951 |
| **8** | days_used | 15.423886 |
| **32** | brand_name_Others | 10.686433 |
| **10** | brand_name_Apple | 10.288252 |
| **44** | os_iOS | 9.381205 |
| **1** | main_camera_mp | 8.834064 |
| **35** | brand_name_Samsung | 7.891218 |

| | feature | VIF |
|---|---|---|
| **45** | 4g_yes | 6.424762 |
| **19** | brand_name_Huawei | 5.826002 |
| **22** | brand_name_LG | 4.894792 |
| **2** | selfie_camera_mp | 4.518138 |
| **24** | brand_name_Lenovo | 4.359097 |
| **41** | brand_name_ZTE | 3.745746 |
| **31** | brand_name_Oppo | 3.678166 |
| **40** | brand_name_Xiaomi | 3.552834 |
| **29** | brand_name_Nokia | 3.507163 |
| **28** | brand_name_Motorola | 3.357255 |
| **38** | brand_name_Vivo | 3.357235 |
| **11** | brand_name_Asus | 3.338550 |
| **26** | brand_name_Micromax | 3.328745 |
| **18** | brand_name_Honor | 3.298441 |
| **17** | brand_name_HTC | 3.297498 |
| **9** | brand_name_Alcatel | 2.873169 |
| **36** | brand_name_Sony | 2.556546 |
| **25** | brand_name_Meizu | 2.377896 |
| **15** | brand_name_Gionee | 2.156928 |
| **33** | brand_name_Panasonic | 1.971641 |
| **34** | brand_name_Realme | 1.943801 |
| **46** | 5g_yes | 1.910692 |
| **39** | brand_name_XOLO | 1.894989 |
| **42** | os_Others | 1.790860 |
| **3** | int_memory | 1.730913 |
| **13** | brand_name_Celkon | 1.726589 |
| **27** | brand_name_Microsoft | 1.700498 |
| **37** | brand_name_Spice | 1.630794 |
| **43** | os_Windows | 1.627687 |
| **12** | brand_name_BlackBerry | 1.624200 |

| | feature | VIF |
|---|---|---|
| **23** | brand_name_Lava | 1.601250 |
| **21** | brand_name_Karbonn | 1.540543 |
| **30** | brand_name_OnePlus | 1.488148 |
| **14** | brand_name_Coolpad | 1.344697 |
| **20** | brand_name_Infinix | 1.330232 |
| **16** | brand_name_Google | 1.289148 |

In [49]:
```
# creating function to treat multicollineraity in any model
def treating_multicollinearity(predictors, target, high_vif_columns):

    adj_r2 = []
    rmse = []


    for cols in high_vif_columns:
        train = predictors.loc[:, ~predictors.columns.str.startswith(cols)]
        olsmodel = sm.OLS(target, train).fit()
        adj_r2.append(olsmodel.rsquared_adj)
        rmse.append(np.sqrt(olsmodel.mse_resid))


    temp = pd.DataFrame(
        {
            "col": high_vif_columns,
            "Adj. R-squared after_dropping col": adj_r2,
            "RMSE after dropping col": rmse,
        }
    ).sort_values(by="Adj. R-squared after_dropping col", ascending=False)
    temp.reset_index(drop=True, inplace=True)

    return temp
```

In [50]:
```
def checking_vif(df_features):

    features_for_vif = df_features.drop(columns=['const']) if 'const' in
df_features.columns else df_features

    vif_data = pd.DataFrame()
    vif_data['feature'] = features_for_vif.columns
    vif_data['VIF'] = [variance_inflation_factor(features_for_vif.values, i)
for i in range(features_for_vif.shape[1])]

    return vif_data.sort_values(by='VIF', ascending=False)
```

In [51]:
```
col_list = ['release_year','screen_size',
'weight','battery','ram','days_used','brand_name_Others','brand_name_Apple','o
s_IOS','main_camera_mp','brand_name_Samsung','4g_yes','brand_name_Huawei']
```

```
res = treating_multicollinearity(X_train, y_train, col_list)
res
```

Out[51]:

| | col | Adj. R-squared after_dropping col | RMSE after dropping col |
|---|---|---|---|
| **0** | brand_name_Others | 0.757218 | 0.283403 |
| **1** | brand_name_Huawei | 0.757217 | 0.283404 |
| **2** | days_used | 0.757131 | 0.283454 |
| **3** | battery | 0.757122 | 0.283459 |
| **4** | os_IOS | 0.757116 | 0.283463 |
| **5** | brand_name_Samsung | 0.757079 | 0.283484 |
| **6** | brand_name_Apple | 0.756832 | 0.283629 |
| **7** | release_year | 0.755339 | 0.284498 |
| **8** | weight | 0.753293 | 0.285685 |
| **9** | 4g_yes | 0.750196 | 0.287473 |
| **10** | screen_size | 0.746758 | 0.289444 |
| **11** | ram | 0.744070 | 0.290976 |
| **12** | main_camera_mp | 0.701137 | 0.314436 |

In [52]:
```
# removed brand_name_others column and reduced the vif scores but still way
above 5 but cannot get rid of important high vif columns
col_to_drop = 'brand_name_Others'
x_train2 = X_train.loc[:, ~X_train.columns.str.startswith(col_to_drop)]
x_test2 = X_test.loc[:, ~X_test.columns.str.startswith(col_to_drop)]

#  checking vif
vif = checking_vif(x_train2)
print("VIF after dropping ", col_to_drop)
display(vif)
```

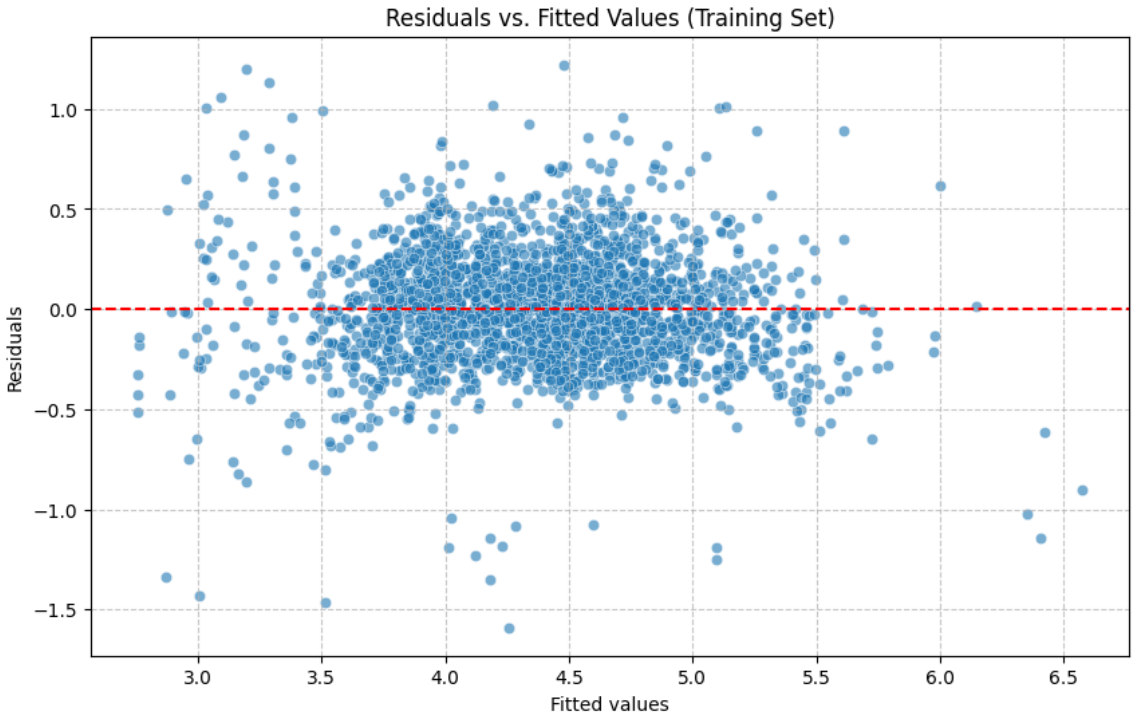
```
VIF after dropping  brand_name_Others
```

| | feature | VIF |
|---|---|---|
| **0** | screen_size | 93.794660 |
| **7** | release_year | 74.058806 |
| **6** | weight | 28.685257 |
| **5** | battery | 26.873320 |

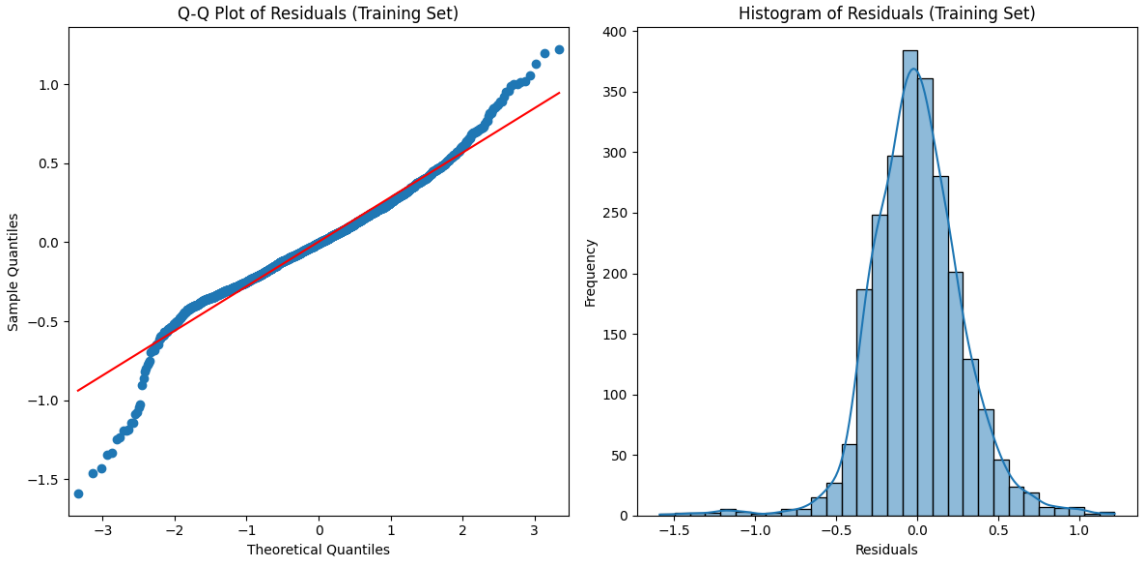
| | feature | VIF |
|---|---|---|
| **4** | ram | 21.132605 |
| **8** | days_used | 15.409631 |
| **10** | brand_name_Apple | 9.709983 |
| **43** | os_iOS | 9.380006 |
| **1** | main_camera_mp | 8.832987 |
| **44** | 4g_yes | 6.424096 |
| **2** | selfie_camera_mp | 4.517578 |
| **45** | 5g_yes | 1.910686 |
| **41** | os_Others | 1.788931 |
| **34** | brand_name_Samsung | 1.730385 |
| **3** | int_memory | 1.730296 |
| **42** | os_Windows | 1.625359 |
| **19** | brand_name_Huawei | 1.584969 |
| **29** | brand_name_Nokia | 1.539202 |
| **27** | brand_name_Microsoft | 1.430330 |
| **22** | brand_name_LG | 1.423361 |
| **31** | brand_name_Oppo | 1.395627 |
| **37** | brand_name_Vivo | 1.359718 |
| **39** | brand_name_Xiaomi | 1.349657 |
| **24** | brand_name_Lenovo | 1.348875 |
| **18** | brand_name_Honor | 1.342804 |
| **40** | brand_name_ZTE | 1.315313 |
| **28** | brand_name_Motorola | 1.288277 |
| **17** | brand_name_HTC | 1.251117 |
| **26** | brand_name_Micromax | 1.249666 |
| **11** | brand_name_Asus | 1.245657 |
| **35** | brand_name_Sony | 1.207482 |
| **9** | brand_name_Alcatel | 1.202542 |
| **25** | brand_name_Meizu | 1.178348 |
| **12** | brand_name_BlackBerry | 1.174073 |

| | feature | VIF |
|---|---|---|
| **33** | brand_name_Realme | 1.166387 |
| **13** | brand_name_Celkon | 1.144722 |
| **15** | brand_name_Gionee | 1.126589 |
| **32** | brand_name_Panasonic | 1.106023 |
| **38** | brand_name_XOLO | 1.105307 |
| **30** | brand_name_OnePlus | 1.090893 |
| **23** | brand_name_Lava | 1.079312 |
| **20** | brand_name_Infinix | 1.079115 |
| **36** | brand_name_Spice | 1.078959 |
| **21** | brand_name_Karbonn | 1.069451 |
| **14** | brand_name_Coolpad | 1.045545 |
| **16** | brand_name_Google | 1.039233 |

In [53]:
```
# No pattern in variance so homoscedasticity and linearity assumptions pass

# Get predicted values and residuals for the training set
y_pred_train = model.predict(X_train)
residuals_train = y_train - y_pred_train

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_pred_train, y=residuals_train, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Fitted values')
plt.ylabel('Residuals')
plt.title('Residuals vs. Fitted Values (Training Set)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```




In [54]:
```
#Checking normailty, model is normal
plt.figure(figsize=(12, 6))

# Q-Q plot
plt.subplot(1, 2, 1)
sm.qqplot(residuals_train, line='s', ax=plt.gca())
plt.title('Q-Q Plot of Residuals (Training Set)')

# histogram of residuals
plt.subplot(1, 2, 2)
sns.histplot(residuals_train, kde=True, bins=30)
plt.title('Histogram of Residuals (Training Set)')
plt.xlabel('Residuals')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```




The Durbin-Watson statistic is 1.709. This value is close to 2, suggesting that there is no significant autocorrelation in the residuals, which is a good sign for the independence assumption.

## Final Model

In [55]:
```
#76% of variance in data is still explained but has slightly decreased
finalmodel = sm.OLS(y_train, x_train2).fit()
finalmodel.summary()
```

Out[55]:

OLS Regression Results

| | | | |
|---|---|---|---|
| **Dep. Variable:** | normalized_used_price | **R-squared:** | 0.762 |
| **Model:** | OLS | **Adj. R-squared:** | 0.757 |
| **Method:** | Least Squares | **F-statistic:** | 164.8 |
| **Date:** | Sat, 11 Jul 2026 | **Prob (F-statistic):** | 0.00 |
| **Time:** | 18:28:19 | **Log-Likelihood:** | -358.28 |
| **No. Observations:** | 2417 | **AIC:** | 810.6 |
| **Df Residuals:** | 2370 | **BIC:** | 1083. |
| **Df Model:** | 46 | | |
| **Covariance Type:** | nonrobust | | |

| | coef | std err | t | P>\|t\| | [0.025 | 0.975] |
|---|---|---|---|---|---|---|
| **const** | 48.4010 | 10.665 | 4.538 | 0.000 | 27.488 | 69.314 |
| **screen_size** | 0.0400 | 0.004 | 10.147 | 0.000 | 0.032 | 0.048 |

| | | | | | | |
|---|---|---|---|---|---|---|
| **main_camera_mp** | 0.0383 | 0.002 | 23.400 | 0.000 | 0.035 | 0.042 |
| **selfie_camera_mp** | 0.0226 | 0.001 | 16.904 | 0.000 | 0.020 | 0.025 |
| **int_memory** | 0.0004 | 7.57e-05 | 5.115 | 0.000 | 0.000 | 0.001 |
| **ram** | 0.0706 | 0.006 | 11.329 | 0.000 | 0.058 | 0.083 |
| **battery** | 8.619e-06 | 8.9e-06 | 0.968 | 0.333 | -8.83e-06 | 2.61e-05 |
| **weight** | 0.0009 | 0.000 | 6.199 | 0.000 | 0.001 | 0.001 |
| **release_year** | -0.0227 | 0.005 | -4.285 | 0.000 | -0.033 | -0.012 |
| **days_used** | -3.402e-05 | 3.67e-05 | -0.927 | 0.354 | -0.000 | 3.8e-05 |
| **brand_name_Alcatel** | -0.0317 | 0.038 | -0.842 | 0.400 | -0.106 | 0.042 |
| **brand_name_Apple** | 0.3453 | 0.173 | 1.993 | 0.046 | 0.006 | 0.685 |
| **brand_name_Asus** | 0.0891 | 0.034 | 2.592 | 0.010 | 0.022 | 0.157 |
| **brand_name_BlackBerry** | 0.0757 | 0.073 | 1.044 | 0.297 | -0.066 | 0.218 |
| **brand_name_Celkon** | -0.3114 | 0.063 | -4.913 | 0.000 | -0.436 | -0.187 |
| **brand_name_Coolpad** | 0.0133 | 0.084 | 0.159 | 0.874 | -0.151 | 0.177 |
| **brand_name_Gionee** | 0.0733 | 0.047 | 1.557 | 0.120 | -0.019 | 0.166 |
| **brand_name_Google** | 0.3395 | 0.091 | 3.714 | 0.000 | 0.160 | 0.519 |
| **brand_name_HTC** | 0.0470 | 0.035 | 1.338 | 0.181 | -0.022 | 0.116 |
| **brand_name_Honor** | -0.0852 | 0.037 | -2.304 | 0.021 | -0.158 | -0.013 |
| **brand_name_Huawei** | 0.0048 | 0.027 | 0.176 | 0.860 | -0.049 | 0.058 |
| **brand_name_Infinix** | 0.0090 | 0.093 | 0.097 | 0.923 | -0.174 | 0.192 |
| **brand_name_Karbonn** | -0.1034 | 0.067 | -1.536 | 0.125 | -0.235 | 0.029 |
| **brand_name_LG** | 0.0512 | 0.029 | 1.786 | 0.074 | -0.005 | 0.108 |
| **brand_name_Lava** | -0.0929 | 0.064 | -1.445 | 0.148 | -0.219 | 0.033 |
| **brand_name_Lenovo** | -0.0385 | 0.030 | -1.290 | 0.197 | -0.097 | 0.020 |
| **brand_name_Meizu** | 0.0291 | 0.044 | 0.656 | 0.512 | -0.058 | 0.116 |
| **brand_name_Micromax** | -0.1215 | 0.035 | -3.513 | 0.000 | -0.189 | -0.054 |
| **brand_name_Microsoft** | 0.0375 | 0.098 | 0.382 | 0.702 | -0.155 | 0.230 |
| **brand_name_Motorola** | -0.0681 | 0.036 | -1.916 | 0.055 | -0.138 | 0.002 |
| **brand_name_Nokia** | 0.1303 | 0.039 | 3.301 | 0.001 | 0.053 | 0.208 |

| | | | | | | |
|---|---|---|---|---|---|---|
| **brand_name_OnePlus** | 0.2444 | 0.074 | 3.302 | 0.001 | 0.099 | 0.389 |
| **brand_name_Oppo** | 0.0743 | 0.035 | 2.115 | 0.035 | 0.005 | 0.143 |
| **brand_name_Panasonic** | -0.0202 | 0.050 | -0.401 | 0.688 | -0.119 | 0.079 |
| **brand_name_Realme** | 0.0814 | 0.055 | 1.476 | 0.140 | -0.027 | 0.189 |
| **brand_name_Samsung** | 0.0574 | 0.024 | 2.425 | 0.015 | 0.011 | 0.104 |
| **brand_name_Sony** | -0.0264 | 0.042 | -0.623 | 0.533 | -0.110 | 0.057 |
| **brand_name_Spice** | -0.2241 | 0.063 | -3.568 | 0.000 | -0.347 | -0.101 |
| **brand_name_Vivo** | 0.0077 | 0.037 | 0.207 | 0.836 | -0.065 | 0.080 |
| **brand_name_XOLO** | -0.0506 | 0.053 | -0.960 | 0.337 | -0.154 | 0.053 |
| **brand_name_Xiaomi** | 0.0459 | 0.035 | 1.309 | 0.191 | -0.023 | 0.115 |
| **brand_name_ZTE** | -0.0381 | 0.033 | -1.156 | 0.248 | -0.103 | 0.027 |
| **os_Others** | -0.1485 | 0.040 | -3.736 | 0.000 | -0.226 | -0.071 |
| **os_Windows** | -0.0460 | 0.054 | -0.860 | 0.390 | -0.151 | 0.059 |
| **os_iOS** | -0.1022 | 0.181 | -0.565 | 0.572 | -0.457 | 0.253 |
| **4g_yes** | 0.1566 | 0.019 | 8.280 | 0.000 | 0.120 | 0.194 |
| **5g_yes** | 0.1268 | 0.037 | 3.440 | 0.001 | 0.055 | 0.199 |

| | | | |
|---|---|---|---|
| **Omnibus:** | 179.059 | **Durbin-Watson:** | 2.008 |
| **Prob(Omnibus):** | 0.000 | **Jarque-Bera (JB):** | 905.127 |
| **Skew:** | -0.102 | **Prob(JB):** | 2.85e-197 |
| **Kurtosis:** | 5.991 | **Cond. No.** | 7.32e+06 |

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 7.32e+06. This might indicate that there are strong multicollinearity or other numerical problems.

In [56]:
```
print("Final Test Performance\n")
testperf2 = model_performance_regression(finalmodel,x_test2,y_test)
testperf2
```

```
Final Test Performance
```

Out[56]:

| | RMSE | MAE | R-squared | Adj. R-squared | MAPE |
|---|---|---|---|---|---|
| **0** | 0.298736 | 0.222669 | 0.76732 | 0.756263 | 5.470802 |

In [57]:
```
print("Final Train Performance\n")
trainperf2 = model_performance_regression(finalmodel,x_train2,y_train)
trainperf2
```

```
Final Train Performance
```

Out[57]:

| | RMSE | MAE | R-squared | Adj. R-squared | MAPE |
|---|---|---|---|---|---|
| **0** | 0.280634 | 0.208421 | 0.761841 | 0.757116 | 4.990579 |

# Actionable Insights and Recommendations

-

## Actionable Insights:

1. Price Distribution: Both `normalized_used_price` and `normalized_new_price` distributions are somewhat normally distributed, but with a slight left skew, suggesting that most devices fall within a common price range, with fewer high-priced items.
2. OS Dominance: Android devices dominate over 93% of the used device market, indicating a large potential customer base for these devices.
3. RAM and Camera: The amount of RAM and camera megapixels are significant factors influencing device prices. Newer models and those supporting 4G/5G generally have higher prices.
4. Correlations: `normalized_new_price` is highly correlated with `normalized_used_price`. Other strong positive correlations with used price include `screen_size`, `main_camera_mp`, `selfie_camera_mp`, `int_memory`, `ram`, `battery`, and `weight`. `release_year` and `days_used` have an inverse relationship with used price, meaning newer devices and those used for fewer days are more expensive.
5. Model Performance: The linear regression model explains approximately 76% of the variance in `normalized_used_price` which is a good baseline for prediction.
6. Significant Predictors: Features such as `screen_size`, `main_camera_mp`, `selfie_camera_mp`, `int_memory`, `ram`, `weight`, `release_year`, `4g_yes`, and `5g_yes` are statistically significant in predicting the used device price. `battery` and `days_used` had less statistical significance in the final model.

## Recommendations:

1. Sourcing & Inventory: Prioritize acquiring and stocking newer model Android devices, especially those with larger screen sizes, higher RAM, better main and selfie cameras, and 4G/5G connectivity, as these features are strong predictors of higher used prices.
2. Marketing In Demand Features: When selling, emphasize the `screen_size`, `RAM`, 4G/5G, and camera specifications (`main_camera_mp`, `selfie_camera_mp`) as these are highly valued by customers and contribute significantly to the selling price.
3. Brand Marketing: Use brand-level insights to target certain customer segments. For example if a brand offers high RAM, these could be marketed towards performance interested consumers.
4. Model Improvement: Continuously monitor the model's performance with new data. Fix problems that come with new data.

---